Masters Degree Project

Bridging a proprietary embedded toolchain to Visual Studio Code

- A case study of PLUS+1 GUIDE



Author(s): Razan Kngo, Marios Gkikopouli
Supervisor(s): Nadeem Abbas
Semester: VT 2026
Course: 4DV50E/5DT90E
Subject: Computer Science

Abstract

Embedded software engineers working with proprietary, safety-critical toolchains often lack adequate editor support, pushing them to juggle between the toolchain for project management and a separate editor for writing C/C++ code. This thesis examines that problem in the context of the PLUS+1 GUIDE environment from Danfoss, using a three-cycle Design Science Research (DSR) methodology. Interviews with ten engineers revealed that missing real-time error checking, slow navigation, and long build times had led many to construct an informal workaround—a second editor running in parallel with manual file synchronisation, effectively a self-assembled shadow IDE. We formalised this workaround by building a proof-of-concept Visual Studio Code extension. Since corrupting the proprietary `.plx` project file was the primary risk, the extension was designed as a lightweight surgical wrapper: it reads the project file, configures standard C/C++ tooling (clangd), and surfaces build and debug commands inside VS Code without touching the certified toolchain. Proprietary graphical editing such as POUs was excluded from scope. A task-based evaluation with five engineers produced an average System Usability Scale score of 73.5 (median 77.5), and four of the five participants said they would adopt the extension given improvements to full C++ support and POU interface editing. The work documents an architectural approach to wrapping a proprietary project format, provides empirical evidence of editor-side productivity barriers, and offers a practical reference for embedded toolchain modernisation.

Keywords: embedded systems, toolchain integration, Visual Studio Code, technical debt, industrial case study

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 The PLUS+1 GUIDE Environment . . . 1
- 1.3 Problem Formulation . . . 2
- 1.4 Motivation . . . 3
- 1.5 Objectives . . . 4
- 1.6 Contributions of the Work . . . 4
- 1.7 Scope and Limitations . . . 5
- 1.8 Thesis Structure . . . 5

2 Related Work **7**
- 2.1 Review of Existing Research . . . 7
  - 2.1.1 Embedded Software Engineering and Toolchain Challenges . . . 7
  - 2.1.2 Editor Integration and Language Services . . . 8
  - 2.1.3 IDE Usability in Industrial and Safety-Critical Settings . . . 9
  - 2.1.4 Security of VS Code Extensions . . . 9
- 2.2 Comparison of Approaches . . . 9
- 2.3 Identified Gaps and Positioning of this Work . . . 10

3 Methodology **12**
- 3.1 Research Approach . . . 12
- 3.2 Data Collection . . . 14
- 3.3 Problem Investigation Methods (RQ1 — Primary) . . . 14
  - 3.3.1 Semi-Structured Interviews . . . 14
  - 3.3.2 Self-Completion Questionnaire . . . 15
  - 3.3.3 Documentation Analysis . . . 16
- 3.4 Solution Design Methods (RQ2) . . . 16
  - 3.4.1 Focused Background Review (Cycle I — Secondary) . . . 16
  - 3.4.2 Weekly Design Walkthroughs (Cycle II — Primary) . . . 17
  - 3.4.3 Iterative Prototyping (Cycle II — Primary) . . . 17
  - 3.4.4 Lightweight Architectural Trade-off Analysis (Cycle II — Primary) 17
- 3.5 Evaluation Methods (RQ3) . . . 18
  - 3.5.1 Initial Usability Probe (Cycle I — Light) . . . 18
  - 3.5.2 Informal Practitioner Feedback Sessions (Cycle II — Informal) . 18
  - 3.5.3 Cycle III Remote Evaluation Questionnaire (Primary) . . . 18
- 3.6 Data Analysis . . . 20
- 3.7 Ethical Considerations . . . 22

4 Results and Analysis **23**
- 4.1 Results . . . 23
  - 4.1.1 Cycle I – Problem Investigation (RQ1) . . . 23
  - 4.1.2 Cycles I–II – Design Artefact (RQ2) . . . 24
  - 4.1.3 Cycle III – Evaluation (RQ3) . . . 27
    - 4.1.3.1 Task Outcomes . . . 27
    - 4.1.3.2 System Usability Scale . . . 28

4.1.3.3 Structured Comparison and Adoption Intent 29
4.2 Analysis 29
4.2.1 RQ1 – Productivity Limitations 29
4.2.2 RQ2 – Solution Design 30
4.2.3 RQ3 – Evaluation 30
4.2.4 Consistency Between Cycle I and Cycle III Responses 31

5 Discussion **32**
5.1 Significance of the Findings 32
5.2 Comparison with the Literature 32
5.3 Limitations 33
5.3.1 Validity Threats 35

6 Conclusion and Future Work **37**
6.1 Conclusion 37
6.2 Future Work 38

Bibliography **40**

A Participant Information Sheet and Consent **42**

B Consent Forms **43**

C Interview Guide **44**

D Self-Completion Questionnaire **48**

E Cycle I Requirements and Pain-Point Evidence **51**

F Cycle I and Cycle II Informal Feedback Items **66**

G Cycle III SUS Item-Level Results **68**

H Cycle III Structured Comparison Responses **69**

1. Introduction

This chapter introduces the research context, motivation, and objectives of the study. Embedded systems engineers often work between two different kinds of tools: proprietary development environments tied to specialised hardware and certified workflows, and modern editors built for flexible software development. The difference becomes clear when engineers use one environment for project configuration and compilation, and another for writing and maintaining source code. This thesis studies that problem through an industrial case and asks whether an integration layer between a proprietary embedded development environment and a modern editor can reduce friction and improve C/C++ development work.

1.1 Background

Embedded software differs from many IT systems in that it must meet strict requirements around timing, safety, and reliability, while running under tight resource and hardware constraints [1]. These conditions have historically led to the use of specialised modelling environments, vendor-specific languages, and certified toolchains rather than the standardised platforms common in other areas of software engineering [1].

Within such environments, project configuration, hardware dependencies, compiler settings, and source-file organisation are often stored in structured project files that serve as the main representation of the system. These files do more than store data: they define how source code is compiled and linked. For that reason, the surrounding development environment is not just a text editor but a workbench for modelling, configuration, and implementation.

However, embedded development environments often provide weak support for capabilities that are standard in mainstream software engineering, including advanced code completion, static analysis, automated testing, version-control integration, and AI-assisted development [2]. This leaves a gap between environments built for platform integration and editors built for day-to-day development work.

At the same time, much of the software industry has moved toward lightweight, extensible editors, especially Visual Studio Code, which combines extensions, automation, version-control integration, and AI-assisted development features [3, 4]. Many embedded and model-driven environments are only loosely connected to this ecosystem because their project structure is encoded in proprietary or specialised configuration formats that general-purpose editors cannot read directly [5]. In such settings, replacing the specialised environment is often unrealistic because of cost, validation demands, and dependence on vendor-specific project semantics. This creates a need for approaches that can improve the editing workflow while preserving the existing embedded toolchain.

1.2 The PLUS+1 GUIDE Environment

The industrial case in this thesis is PLUS+1 GUIDE, a proprietary development environment from Danfoss Power Solutions. Danfoss Power Solutions produces electronic control systems for electrical and hydraulic propulsion and work functions in off-highway vehicles, and PLUS+1 GUIDE supports the design, testing, and simulation of embedded control applications for these systems [6].

Applications built in PLUS+1 GUIDE run on PLUS+1 compliant hardware—controllers, displays, joysticks, sensors, and related electronic control units—used in heavy off-road equipment such as tractors, cranes, and harvesters. Because these machines operate in contexts where software faults may contribute to personal injury or equipment damage, GUIDE is not merely a programming editor. It is part of a safety-critical, proprietary toolchain in which engineers must configure control logic, build in fault checking, and restrict access to operating parameters.

Engineers construct applications through a drag-and-drop interface using logical components and multiple programming languages. Supported languages include the native GUIDE Dataflow language, the five standard IEC 61131 PLC languages, and C/C++, which can be mixed within a single application. This mixed-language model is important for the present study: although GUIDE manages the overall application structure, engineers still need to read, write, navigate, and maintain textual C/C++ code inside projects whose structure is defined by GUIDE-specific project files. The compiled application is then downloaded to target hardware via a companion tool called the PLUS+1 Service Tool, which engineers also use to monitor machine values, log diagnostic data, and tune controller performance [6].

1.3 Problem Formulation

The core problem addressed by this thesis is the *productivity barrier* faced by embedded software engineers who develop C/C++ code in proprietary, domain-specific development environments such as PLUS+1 GUIDE. These environments often handle platform configuration and compilation well, but give limited access to capabilities common in mainstream software engineering, such as code completion, integrated static analysis, AI-assisted development, and smooth version-control workflows. When engineers must switch between a domain-specific environment for project management and a separate editor for implementation work, they face extra coordination overhead, more context switching, and a higher risk that project state and source-code state drift apart.

At a technical level, this problem comes from *semantic incompatibility*. In many embedded toolchains, hardware configuration, compilation parameters, source-file membership, and other project-level details are encoded in specialised structured files that a general-purpose editor cannot read directly. Consequently, standard editor tooling cannot reliably resolve file membership, project structure, or build context without some form of translation, and language services need either extracted compiler context or a workspace-level template approximating the toolchain dialect. Replacing the specialised environment outright would often be too costly, too risky, or too time-consuming for an industrial setting in which existing build behaviour must be preserved. Addressing this gap therefore points toward an integration layer that can present project structure in a form that a modern editor and existing C/C++ tooling can use, while leaving the original project file intact.

At a research level, prior work has shown that modern editor support can be extended to graphical modelling languages [7] and formal specification languages [8]. However, published solutions for wrapping specialised embedded project formats so that engineers can work on C/C++ code in a modern editor are still limited. The problem of connecting a proprietary, safety-critical project representation to a modern editor without replacing the underlying toolchain has received little attention in the literature.

The research questions that structure this thesis are:

RQ1 (Problem Investigation) What are the specific productivity limitations and inef-

ficiencies that embedded software engineers experience when developing C/C++ code in proprietary, domain-specific embedded environments compared to modern development tools?

RQ2 (Solution Design) Which architectural approaches and design solutions can effectively connect specialised project representations with modern editor environments, and what are the trade-offs of different approaches?

RQ3 (Evaluation) To what extent does a case-specific editor integration address the identified productivity limitations, and how do engineers perceive its usability and value compared to the native workflow?

1.4 Motivation

This research is motivated by three concerns: industrial productivity, technical opportunity, and academic relevance. A fourth, empirically grounded dimension emerged during the early phases of the study.

Developer Experience Gap. Embedded systems development often relies on specialised toolchains and IDEs that are less flexible than modern development environments [2, 4]. If these environments can work alongside a modern editor, engineers can reuse familiar workflows, extensions, and AI-assisted tools instead of adapting to a restricted editing context for each platform-specific task. This makes approaches that improve the editing experience without replacing embedded tooling especially relevant, since such tooling is often expensive to change, closely tied to existing processes, or costly to revalidate. AI-assisted development tools can automate repetitive coding tasks, support debugging and testing, and improve software quality [9]. Editor integration mechanisms such as language services, task automation, and standard editor APIs can support the same goal by enabling automation, tooling integration, and more consistent development environments [4, 5, 7, 8]. Early observations also suggested that many engineers had already constructed an informal "shadow IDE" by keeping a second editor open alongside GUIDE, indicating that this cross-editor workflow partly existed before the research artefact was designed.

Technical Debt and Toolchain Infrastructure. Technical debt, including infrastructure issues in the development environment, is a cost that practitioners, especially managers, tend to underestimate [10]. In safety-critical domains governed by standards such as IEC 61508, development toolchains operate under certification constraints that limit how freely they can be changed [1]. The present thesis does not address the certification process itself; the scope is limited to the editor layer, which sits outside the certified build pipeline. This complexity adds to technical debt in the development environment, which has been linked to measurable productivity loss [10]. From this perspective, improvements at the editing layer are relevant because they may reduce development-environment debt without requiring changes to the certified build pipeline. Such changes are also more realistic than full toolchain replacement, which would require much more time, money, retraining, and organisational effort.

Scientific Contribution. The problem of modernising embedded software toolchains has only been partly addressed in the literature on modern editor integration. Existing

research on editor support for non-mainstream languages focuses on formal specification languages and domain-specific languages (DSLs) embedded in modelling tools [5, 8, 11], not on proprietary, XML-project-based embedded toolchains where C/C++ editing depends on project information maintained by an existing environment. This thesis contributes an empirically grounded case study, a concrete integration design, and a set of lessons learned from that design in one embedded-domain setting.

1.5 Objectives

The following objectives guide the research and are aligned with the three research questions:

O1	Investigate and characterise the productivity limitations of C/C++ editing within a proprietary embedded development environment through semi-structured interviews and a self-completion questionnaire with practising engineers (addresses RQ1).
O2	Identify and evaluate architectural approaches for wrapping a specialised structured project format so that it can work with a modern editor and its existing C/C++ tooling, including a comparison of the design trade-offs between alternative integration strategies (addresses RQ2).
O3	Develop and build the selected architectural approach as a proof-of-concept editor integration that supports C/C++ project navigation, configuration of existing language tooling, and build and debug initiation within a modern editor (addresses RQ2).
O4	Evaluate the prototype through a questionnaire-based study with embedded engineers, using self-reported task outcomes, perceived usability ratings, and open-ended reflections to assess perceived usefulness and adoption value (addresses RQ3).

The expected outcomes include a functional software artefact, a validated problem characterisation, a documented integration design rationale, and a set of practical recommendations for further development.

1.6 Contributions of the Work

This thesis makes contributions at three levels:

Software Artefact. The thesis develops a functional proof-of-concept editor integration that parses a specialised project format, configures existing C/C++ editing support, exposes project structure within a modern editor, and provides build and debug integration through the editor's task-execution and launch facilities. The artefact formalises a workflow that practitioners had already begun to construct informally (a second editor running alongside GUIDE, with manual save-and-reload before each build) rather than introducing an entirely new pattern of work. This framing is grounded in the Cycle I evidence and is revisited in Chapter 5.

Empirical Problem Characterisation. The Cycle I investigation (RQ1) produces a catalogue of productivity barriers in a proprietary embedded C/C++ development work-

flow, grounded in semi-structured interviews and a self-completion questionnaire from ten practising engineers. A notable finding from this catalogue is the prevalence of an informal "shadow IDE" already constructed by practitioners, in which a modern editor is run alongside the specialised environment with manual save-and-reload before each build; the Cycle II artefact formalises this existing workaround rather than introducing a new pattern of work.

Evaluation Evidence. The Cycle III questionnaire-based evaluation (RQ3) provides empirical evidence on engineers' perceived usefulness, usability, and adoption concerns for a hybrid workflow that separates project management from source-code editing. Findings are reported as case-level evidence from one industrial setting; transferability to other embedded toolchains is treated as a question of mechanism rather than of measured replication (Chapter 5).

1.7 Scope and Limitations

The scope of this thesis is bounded as follows:

- The extension targets C/C++ code management within a proprietary embedded project environment. Other languages supported by the host environment, including graphical or PLC-oriented notations, are outside the scope of this work.
- The prototype is a proof-of-concept and is not intended for production deployment. It will not undergo the certification process required for safety-critical use under IEC 61508 or ISO 13849.
- The usability evaluation is conducted with a small sample of practising engineers from the case organisation. The results are not intended to be statistically generalisable but to provide actionable, qualitative-plus-quantitative insights for the design cycle.
- Modern editor extension security, including cross-extension data exposure and privilege management [12, 13], is acknowledged as a relevant concern for industrial deployment but is not a primary focus of this thesis. These risks are discussed in Chapter 5 as directions for future hardening.

1.8 Thesis Structure

The remainder of this thesis is organised as follows:

- **Chapter 2 – Related Work:** Reviews existing research across three areas: embedded software engineering and toolchain challenges; editor integration and language services; and IDE usability in industrial settings. Security concerns for modern editor extensions are also surveyed.
- **Chapter 3 – Methodology:** Describes the Design Science Research (DSR) methodology adopted, following Knauss's guidelines for constructive master's thesis work in industry [14]. The three iterative DSR cycles and data collection methods for each research question are detailed.

- **Chapter 4 – Results and Analysis:** Presents findings from all three DSR cycles: the problem characterisation from Cycle I, the design decisions from Cycle II, and the usability evaluation data from Cycle III.
- **Chapter 5 – Discussion:** Interprets the results in relation to the research questions, compares the findings with earlier literature, and critically assesses the limitations of the study.
- **Chapter 6 – Conclusion and Future Work:** Summarises the contributions of the thesis and proposes directions for future development and research.

2. Related Work

This chapter surveys the research literature relevant to the three main themes of this thesis: embedded software engineering and the toolchain challenges that motivate this work; editor integration and language services for specialised environments; and IDE usability in industrial and safety-critical settings. A final subsection addresses security considerations for VS Code extensions, which are relevant to the industrial deployment context of this work. The chapter concludes by identifying the research gap that this thesis addresses.

2.1 Review of Existing Research

2.1.1 Embedded Software Engineering and Toolchain Challenges

Embedded software engineering is characterised by constraints that distinguish it sharply from enterprise or web software development. Programmes must operate in real time, within strict memory and power budgets, and to safety integrity levels mandated by standards such as IEC 61508. An early survey of the field noted that embedded software was growing in complexity at a rate that challenged traditional development processes, and identified model-driven development as a promising path toward managing that complexity [1]. More than a decade later, those tensions remain: embedded engineers continue to rely on proprietary toolchains that were built to meet certification requirements but were not designed with developer usability in mind, and practitioners still find it difficult to introduce new solutions into these established processes [1, 9]. Beningo's practitioner reference on embedded software design argues for modern tooling and processes, including static analysis and CI/CD pipelines, as components of quality embedded software development that many teams currently lack [2].

Besker et al. categorise infrastructure issues (including the development environment) among six distinct technical debt types, and show that managers are consistently less aware than developers of how much time this debt consumes in practice [10]. This suggests that toolchain modernisation is not only a usability issue but also a measurable productivity issue.

Nuratch provides a concrete example of IDE extension work in the embedded domain, demonstrating the pattern of connecting embedded firmware workflows with text editors by adding compiler invocation and device programming directly into the editing environment [15]; although that work targets firmware flashing rather than project-level C/C++ editing, it illustrates the same need to bridge embedded toolchains and modern development environments.

A separate strand of practitioner-facing work, distinct from the academic literature surveyed above, takes the form of editor wrappers around embedded toolchains in industrial use. PlatformIO provides a VS Code integration layer for a range of microcontroller toolchains with documented build interfaces and open-source frameworks and SDKs [16]. Vendor wrappers around IAR Embedded Workbench, Keil μVision, and similar proprietary IDEs have been developed both internally by tool vendors and by third parties. These wrappers occupy the same architectural niche as the artefact developed in this thesis: an integration layer that exposes a proprietary or vendor-specific toolchain to a modern editor without replacing the underlying build pipeline. The distinguishing feature of the present case is that the target toolchain's project file is a proprietary, undocumented XML format rather than a documented build interface, which shifts the inte-

gration challenge from build invocation to project-format reconstruction. To the authors' knowledge, this specific class of wrapper — working against a proprietary project file in a safety-critical context — has not been documented in the academic literature, although the broader wrapper pattern is well established in practice.

2.1.2 Editor Integration and Language Services

The Language Server Protocol (LSP) was introduced by Microsoft in 2016 as part of VS Code and subsequently standardised as an open protocol. It defines a JSON-RPC interface between an editor client and a language server, allowing a single server implementation to serve any compliant editor. Bork and Langer provide an extensive academic treatment of LSP, covering its full architecture, the evolution from a VS Code-specific mechanism to a community standard, and its potential for domains beyond programming languages, including web-based modelling tools [4]. Their work makes clear the value of separating language intelligence from the editor, which reduces both the cost of adding editor support for a language and the cost of adding language support to an editor.

Bunder's empirical case study on decoupling language and editor applies LSP to an Entity-DSL and integrates the resulting language server into both Eclipse Theia and Eclipse IDE [5]. His SWOT analysis of LSP identifies strengths (editor independence, reduced implementation effort) that align closely with the motivation for integrating specialised engineering workflows into modern editors, while the weaknesses he identifies (limited expressiveness for non-standard language semantics) point toward the semantic incompatibility challenge described in the problem formulation.

Barros et al. contribute an empirical study of 30 real-world LSP server implementations, identifying common architectural patterns, quality attributes, and engineering pitfalls [11]. Their findings provide grounded guidance for the solution design phase of this thesis (RQ2), offering evidence on which design choices have proven robust in practice. The study covers a diverse set of language servers but includes no examples from embedded or safety-critical toolchain domains, which reinforces the gap this thesis addresses.

Rask et al. present the creation of a VS Code extension for VDM (Vienna Development Method), a formal specification language, using both LSP and the Debug Adapter Protocol (DAP) [7]. Their work demonstrates that specialist languages used by relatively small technical communities can be effectively supported in VS Code through these protocols, and that doing so lowers the adoption barrier for the language. The inclusion of DAP-based debugging also illustrates the depth of vendor-side integration that full debugging support requires, which informs the implementation strategy and constraints for debugging support in the present thesis.

In subsequent work, Rask et al. argue that standard LSP is insufficient for specification and formal languages, which have domain-specific semantic requirements that the core protocol does not cover, and propose a set of conservative extensions [8]. This argument applies equally to specialised embedded environments whose project structures encode hardware configuration and build semantics outside the assumptions of standard LSP. Bridging this gap requires custom adaptation, which is the central design challenge examined in this thesis.

The literature surveyed above engages with editor-language decoupling at two levels: as a protocol whose expressiveness is the object of study (Rask et al.), and as a mechanism through which existing or new language servers are exposed to modern editors (Bunder, Barros et al.). The present thesis engages with this literature at the second level only. It uses an existing C/C++ language server (clangd, which speaks LSP) as a standard

component, rather than designing a new server or extending the protocol. The integration challenge addressed here is upstream of any single protocol: reconstructing enough project and compilation context for standard C/C++ tooling to operate. In the studied GUIDE projects, the `.p1x` file provides project-level metadata and source-file membership, while include paths, preprocessor definitions, and compiler flags must be supplied from other toolchain knowledge or generated build artefacts rather than extracted directly from the `.p1x` XML. The editor-integration literature is therefore relevant as background on the ecosystem and as a source of architectural patterns for decoupling, rather than as the direct technical lineage of the contribution. Whether constructs specific to the host environment would benefit from language-service-level support beyond what an existing C/C++ language server provides is treated as future work in Chapter 6.

2.1.3 IDE Usability in Industrial and Safety-Critical Settings

Wiesmayr et al. conduct a formal user study on 4diac, an open-source IDE for IEC 61131-3 and IEC 61499 visual programming. Their study uses the Cognitive Dimensions of Notations framework to identify usability problems and a structured evaluation with industrial automation engineers to validate their findings [17].

The study shows that even IDEs designed specifically for the target domain introduce usability friction for practising engineers, particularly around abstraction, error proneness, and difficult mental operations [17]. This work is analogous to the evaluation component of this thesis (RQ3): the target users, the domain (IEC 61131-based embedded programming), and the evaluation approach (structured study with industrial practitioners) are closely aligned.

2.1.4 Security of VS Code Extensions

As VS Code has grown to host tens of thousands of publicly available extensions [12], concerns about extension security have attracted academic attention. Edirimannage et al. analyse 52,880 extensions and find that 5.6% exhibit suspicious behaviours that could compromise development environments or leak sensitive information about the developer's product [12]. Liu et al. study cross-extension data exposure, documenting how credentials and access tokens can leak between co-installed extensions through shared VS Code APIs [13]. These findings are relevant to the deployment context of this thesis, which involves commercially sensitive source code and hardware-configuration data, but extension security is not a primary focus of this work and is discussed only in the limitations section in Chapter 5.

2.2 Comparison of Approaches

Table 2.2.1 summarises the key prior works discussed above, comparing them along dimensions most relevant to this thesis: the type of language or toolchain addressed, the approach taken, the IDE target, and the primary limitation identified.
The table shows a consistent pattern: prior work either targets mainstream languages (where the project-format challenge does not arise), DSLs embedded in modelling frameworks, or formal specification languages. The three gaps that follow from this pattern are identified and discussed in the next section.

2.3 Identified Gaps and Positioning of this Work

The review above reveals three distinct gaps in the literature that this thesis addresses:

Gap 1: No modern editor integration solutions for embedded toolchain integration. The literature on modern editor integration has focused on mainstream programming languages, with only isolated work addressing DSLs hosted in modelling frameworks [5, 18, 11]. No prior work directly addresses the integration of a specialised embedded toolchain with heavy project semantics into a modern editor environment for C/C++ work. This matters because the main challenge — bridging semantic incompatibility between a proprietary project representation and existing editor tooling — differs both from the academic case studies above and from industrial wrappers such as PlatformIO, which target toolchains with documented or open build interfaces rather than proprietary project files.

Gap 2: No empirical studies on embedded toolchain modernisation. While Besker et al. provide empirical evidence that technical debt broadly imposes a measurable productivity cost [10], empirical characterisations of the specific problems experienced by engineers working in proprietary embedded IDEs remain rare. The Cycle I investigation of this thesis contributes such a characterisation in an industrial setting.

Gap 3: No documented architectural guidance for project-format-to-editor-tooling integration. Rask et al. [8] argue that standard LSP is insufficient for specification languages, where domain-specific semantics fall outside the protocol's assumptions. The case addressed in this thesis is different in character: the user-authored C source files edited inside the host environment are standard C, but they are embedded in a GUIDE-managed project context that also contains generated wrapper code, generated interfaces, toolchain include paths, preprocessor definitions, and source-file membership rules. Existing C/C++ language servers can support the handwritten C code only to the extent that this surrounding compilation context is reconstructed accurately. In practice, the prototype configured enough context for useful editor support, but it did not fully resolve recognition of all GUIDE-generated code and generated build artefacts. The architectural gap addressed here is therefore not primarily a protocol gap but a *project-format-to-editor-tooling configuration* gap: how to read project membership from the proprietary `.plx` file and combine it with toolchain- and build-derived context in a form that off-the-shelf C/C++ tools (language servers, build tasks, debug adapters) can use, without modifying either the proprietary toolchain or the editor infrastructure. The literature offers no documented guidance for this configuration problem. This thesis contributes a case-specific integration design and trade-off analysis, while treating complete generated-code recognition as an unresolved limitation and future-work item.

This thesis sits at the intersection of three bodies of work that have not previously been brought together: embedded software toolchain challenges, modern editor integration, and industrial IDE usability research. By combining Design Science Research with qualitative inquiry and descriptive quantitative evaluation, it aims to produce findings that are relevant to the case organisation and cautiously useful beyond it.

Study / Example	**Language / Domain**	**Editor**	**Approach**	**Key Limitation / Difference**
Bunder [5]	Entity-DSL	Theia, Eclipse	LSP-based DSL integration	DSL focus; no proprietary project-format adaptation
Barros et al. [11]	30 mainstream LSPs	Various	Comparative study of language-server reuse	No embedded or safety-critical project-format cases
Rask et al. [7]	VDM formal language	VS Code	VS Code extension with LSP and DAP support	Full debugging depends on domain-specific debug-adapter support
Rask et al. [8]	Specification languages	VS Code	Conservative extensions to standard LSP	Addresses language semantics rather than embedded project reconstruction
Wiesmayr et al. [17]	IEC 61499 / 61131	4diac IDE	Cognitive-dimensions usability evaluation	No VS Code integration; no project-format wrapper
Serial Bootloader [15]	Embedded firmware	Text editors	Editor-driven compile and flashing workflow	No project-level semantics; no LSP-based C/C++ context extraction
PlatformIO [16]	Microcontroller development	VS Code	Wrapper around documented embedded build interfaces	Targets documented/open toolchains rather than proprietary project files
Rodriguez et al. [18]	Graphical models	Theia	Web-based graphical modelling environment	Graphical/UML focus; not text-based C/C++ editing
This thesis	**C/C++ in a specialised embedded environment**	**VS Code**	**Project-format-to-editor-tooling wrapper**	**Extracts proprietary project semantics while using existing C/C++ tooling unmodified**

Table 2.2.1: Comparison of existing approaches to language toolchain modernisation.

3. Methodology

The research questions formulated in Chapter 1 are addressed through a methodological framework that integrates problem investigation, solution design, and artefact evaluation into a single iterative structure. The overarching approach is *Design Science Research (DSR)*, applied following the guidelines proposed by Knauss [14] for constructive master's thesis work conducted in collaboration with industry. DSR is a constructive research methodology that focuses on producing a concrete design artefact—in this case, a Visual Studio Code extension—while simultaneously generating knowledge through empirical inquiry [14]. The work is structured around a *regulative cycle* [14] in which problem investigation, solution design, and evaluation are revisited iteratively across three successive cycles, each lasting approximately four to five weeks. Each cycle is internally organised as a build-and-evaluate *Design Cycle* [19], in which artefact construction and artefact evaluation alternate to drive progressive refinement.

The remainder of this chapter is organised as follows. Section 3.1 presents the overall research approach and justifies the choice of DSR in this specific industrial and academic context. Section 3.2 describes the data collection methods employed in each cycle. Section 3.3 presents the instruments used to address RQ1. Section 3.4 presents the methods used to address RQ2, and Section 3.5 presents the methods used to address RQ3. Section 3.6 explains the analysis techniques applied to the collected data. Section 3.7 addresses the ethical considerations governing the involvement of human participants. The chapter closes with a brief methodology summary.

3.1 Research Approach

This research adopts *Design Science Research (DSR)* as its methodological foundation, following the guidelines established by Knauss [14] for constructive master's thesis projects conducted in close partnership with industry. DSR is a constructive research methodology focused on producing a concrete design artefact while simultaneously generating empirical knowledge through structured problem investigation and systematic evaluation [14]. This dual orientation makes DSR well suited to the present study, which must balance practical value to the industrial partner (Danfoss Power Solutions AB) with the academic rigour required of a master's thesis.

The choice of DSR over alternative approaches warrants explicit justification. A purely *experimental* approach would require controlled laboratory conditions and replicable baseline comparisons that cannot be established for a proprietary, domain-specific toolchain. A *case study* approach, in the tradition of Yin [20], would be appropriate for in-depth, exploratory analysis but does not accommodate the constructive orientation required to design and build a software artefact as a primary research output. A *survey-based* approach could characterise developer experience at scale but would lack the depth needed to uncover the technical constraints of the PLUS+1 project format. DSR subsumes elements of all three: problem investigation through qualitative inquiry (addressing RQ1), artefact construction through iterative design (addressing RQ2), and artefact evaluation through structured practitioner studies (addressing RQ3). This structure maps onto the three research questions and onto Knauss's Guideline G3, which recommends that knowledge questions be tied one-to-one to the phases of the regulative cycle [14].

Following Knauss's Guideline G2, the thesis is executed across three iterative DSR cycles, each contributing to all three research questions simultaneously but with a shifting

emphasis: each cycle places its primary focus on one knowledge dimension. Within each cycle, the build-and-evaluate Design Cycle described by Hevner [19] is exercised to ensure that artefact construction and empirical inquiry advance together rather than in sequence. Table 3.1.1 summarises this structure.

Table 3.1.1: DSR Cycle Structure with Shifting Emphasis—adapted from Knauss [14]

Cycle	**RQ1 — Problem**	**RQ2 — Solution**	**RQ3 — Evaluation**
Cycle I (Wks 1–5)	**PRIMARY:** Semi-structured interviews; self-completion questionnaire (equivalent alternative); documentation analysis of `.p1x` files	Secondary: Focused background review of prior research and technical documentation (LSP, VS Code APIs, technical debt, hybrid workflows)	Light: Informal usability probe with 1–2 participants on the initial scaffold
Cycle II (Wks 6–10)	Validation: Use-based reflective question per feedback session; running log of recurring and newly surfaced themes	**PRIMARY:** Weekly design walkthroughs with engineers; lightweight architectural trade-off analysis; iterative prototyping over four implementation increments (I1–I4); Architecture Decision Records (ADRs); integration design documentation	Informal: Practitioner feedback session per increment
Cycle III (Wks 11–15)	Final validation through Cycle III questionnaire items on pain-point resolution (items C1–C8), merged with Cycle II validation evidence	Minor: Architecture documentation refinements; bug triage notes	**PRIMARY:** Asynchronous remote evaluation questionnaire combining a six-task self-directed task battery with per-task self-report (SR1–SR4), the System Usability Scale (SUS), and eight reflective comparison questions (C1–C8)

Table 3.1.1 summarises the shifting emphasis across the three cycles. Cycle I centres on problem investigation, Cycle II translates the problem characterisation into a prototype through iterative design and weekly walkthrough feedback, and Cycle III provides the main artefact evaluation. Cycle III was initially intended to include interview-based evaluation, but participant availability and the remaining project time made an asynchronous remote questionnaire the feasible evaluation instrument. The resulting evidence is collected through the questionnaire responses.

This iterative structure, with its shifting emphasis, follows Knauss's Guideline G5 and serves two functions. First, it requires the design of the artefact to be grounded in a validated problem understanding before substantial implementation effort is committed. Second, it provides multiple feedback loops through which findings from earlier cycles can redirect the work of later ones, a property of the regulative cycle that separates DSR from linear, waterfall-style development [14].

3.2 Data Collection

Data collection is organised by research question rather than by instrument. The instruments used and their cycle placement are summarised in Table 3.1.1 above; the remainder of this chapter describes each instrument in detail.

3.3 Problem Investigation Methods (RQ1 — Primary)

The instruments described in this section together address RQ1. Semi-structured interviews and the self-completion questionnaire collect first-person accounts of the productivity barriers engineers experience, while documentation analysis establishes the technical constraints of the PLUS+1 GUIDE environment. In addition to the Cycle I instruments, later Cycle II feedback sessions and the Cycle III questionnaire include reflective items that revisit the Cycle I pain-point themes. The procedure for coding and synthesising these materials is described in Section 3.6.

3.3.1 Semi-Structured Interviews

Semi-structured interviews are the principal first-person instrument for RQ1. Their purpose is to obtain detailed accounts of the pain points, workarounds, and productivity costs that engineers experience when using PLUS+1 GUIDE for C/C++ development. This format was preferred over a fully structured questionnaire because the problem space is insufficiently understood in advance to permit exhaustive closed-ended items; the semi-structured format allows the interviewer to follow emerging themes through follow-up questions while still covering all planned topics.

Participant selection. Participants are recruited through availability and voluntary interest within the case organisation. Rather than applying a formal sampling strategy with predefined quotas, the researchers invite engineers who are accessible during the study period and willing to contribute their experience with PLUS+1 GUIDE. The practical inclusion criterion is that participants have relevant experience of writing, maintaining, reviewing, or supporting C/C++ development in GUIDE projects, so that their responses are grounded in the workflow under investigation. Because recruitment depends on access, availability, and voluntary participation within one industrial case organisation, the participant group is treated as a purposive convenience sample rather than as a statistically representative sample of all GUIDE users at Danfoss. The final composition of the participant group is reported in Chapter 4.

Session format. Each interview is scheduled for 45–60 minutes and conducted either in person at the Danfoss Älmhult facility or remotely via Microsoft Teams. Participants receive a written information sheet (Appendix A) before each session describing the study purpose, the voluntary nature of their participation, and the intended use of the data. Written or email-confirmed informed consent (Appendix B) is obtained before any recording begins. The interview follows a three-part structure: a warm-up phase establishing the participant's role, years of experience, and typical development tasks; a core phase covering the five thematic areas defined in the interview guide (Appendix C); and a closing phase in which the participant may raise anything not covered.

Thematic coverage. The core phase is organised around five themes derived from the problem formulation in Chapter 1. Theme 1 (Current workflow) captures how engineers navigate, edit, and compile C/C++ code in PLUS+1 GUIDE on a typical day. Theme 2 (Tool limitations) elicits which features engineers find missing from GUIDE's built-in editor and how those gaps manifest in practice. Theme 3 (Existing workarounds) documents external tools, scripts, and ad hoc adaptations engineers have built to compensate for those gaps. Theme 4 (Experience with modern editors) establishes a comparison point with VS Code, JetBrains, or similar editors used outside GUIDE. Theme 5 (Openness to change) probes how engineers would respond to a hybrid workflow that combines GUIDE with a modern editor for C/C++ editing, including any concerns about synchronisation, trust, or team adoption.

3.3.2 Self-Completion Questionnaire

A self-completion questionnaire is offered as an equivalent alternative to the live interview for participants whose schedule, location, or availability does not permit a 45–60 minute session. Self-completion questionnaires are a recognised complement to interviews in qualitative research, especially in applied research settings where time and practitioner availability constrain data collection [21]. In the present study, the questionnaire is not a separate instrument addressing different questions: it covers exactly the same five themes as the interview guide and collects the same kind of free-text response, but in a written, asynchronous form. This design supports comparability between the two collection modes.

Instrument design. The questionnaire is delivered through Google Forms and consists of fifteen open-ended questions distributed across seven sections that mirror the interview structure: background, daily workflow, pain points and limitations, build process, modern tooling, openness to change, and open feedback. Each question is accompanied by short prompt items (*"you may also consider. . . "*) that correspond to the follow-up questions an interviewer would use in a live session. Participants are told that questions may be skipped if they do not apply, and that the level of detail in their responses directly determines how much their input can shape the design of the extension. The full questionnaire is reproduced in Appendix D.

Distribution and consent. The questionnaire is distributed by email to engineers who are reachable during the study period and who express interest in participating, accompanied by the same information sheet (Appendix A) used for interview participants. Submission of the form constitutes consent: the consent statement appears at the top of the form and explicitly states that completing and submitting the form confirms that the participant has read the information sheet, understands that participation is voluntary, and consents to anonymised use of their responses in the thesis and any related publications. No personally identifying information is collected through the form; participants are assigned the same P1, P2, . . . identifier scheme used for interviews, with the identifier allocated by order of submission.

Data handling. Submitted responses are exported from Google Forms to a study spreadsheet and the original form responses are then deleted from Google. The exported data is stored alongside interview transcripts in the same access-controlled folder. Participants are referred to in the thesis simply as P1, P2, . . . regardless of mode.

3.3.3 Documentation Analysis

Documentation analysis examines technical documentation as a supplementary, non-reactive method. Its purpose is to establish a precise technical understanding of the constraints that a VS Code integration must respect, including constraints that may not be fully articulated by engineers in interviews or questionnaire responses because they reside in technical layers that engineers do not routinely reason about explicitly. The method supports the prototype design by clarifying available interfaces, configuration formats, command-line behaviour, and compiler-related assumptions.

Categories of documents. The documentation reviewed consists mainly of technical sources needed to implement the integration: PLUS+1 GUIDE user documentation, especially command-line usage and build-related behaviour; available information about compiler flags and project configuration; the VS Code Extension API documentation relevant to tree views, tasks, commands, workspace handling, and diagnostic presentation; and documentation for existing C/C++ language-service tooling such as `clangd`. These sources are used to understand what information the extension must extract from a GUIDE project and how that information can be translated into editor and language-tool configuration. The review is therefore practical and implementation-oriented, rather than a formal audit of all available company documentation.

Use of documentation. Documentation sources are consulted when they become relevant to concrete design or implementation questions, for example how GUIDE can be invoked from the command line, which compiler flags must be preserved, how VS Code tasks and diagnostics are registered, and how `clangd` expects include paths and preprocessor definitions to be supplied. Where a source contains non-public company or toolchain information, the documents themselves are not reproduced in the thesis.

3.4 Solution Design Methods (RQ2)

The methods used to address RQ2 fall into four groups: a focused background review of prior research and technical documentation, weekly design walkthroughs, four iterative implementation increments, and a lightweight architectural trade-off analysis that connects design decisions to increment feedback and produces the design rationale reported in Chapter 4. The background review is conducted in Cycle I as a secondary RQ2 contribution; the walkthroughs, increments, and trade-off analysis are concentrated in Cycle II, where RQ2 is the primary focus. Cycle III makes only a minor RQ2 contribution through final architecture documentation refinements.

3.4.1 Focused Background Review (Cycle I — Secondary)

A focused background review is conducted in Cycle I for two reasons: to populate Chapter 2 with a positioned discussion of prior work, and to generate an initial set of solution candidates for the Cycle II prototype increments. The review is targeted rather than systematic, drawing from peer-reviewed literature on the Language Server Protocol, VS Code extension architecture, embedded toolchain integration, and developer productivity in specialised environments, supplemented by official technical documentation for VS Code and LSP-related tooling. The date range is primarily 2015–2026, with earlier sources included only when foundational to the research area.

3.4.2 Weekly Design Walkthroughs (Cycle II — Primary)

Weekly design walkthroughs are the main practitioner-facing activity in Cycle II. Their purpose is not to conduct formal architecture workshops or to make architectural decisions jointly with engineers and supervisors. Instead, they provide a regular feedback loop in which the current prototype state is demonstrated, the implementation approach is explained, and engineers can raise practical feedback, concerns, or misunderstandings before the next increment is developed. This format reflects the actual industrial setting: practitioner time is limited, and the most realistic way to involve engineers during implementation is to show working increments regularly and collect focused feedback on whether the behaviour matches their workflow expectations.

The walkthroughs are aligned with the implementation increments described in Section 3.4.3. Each week, the authors prepare a short demonstration of the current state of the extension, explain what was implemented and why, and invite comments on usability, workflow fit, risks, and missing functionality. Feedback is captured as notes rather than as formal workshop transcripts. When a concern affects the design, it is translated into an action item, a refinement of the prototype backlog, or an Architecture Decision Record. In this way, the walkthroughs support Hevner's [19] view of the Design Cycle as an iterative build-and-evaluate loop, while avoiding the incorrect implication that formal architecture workshops were held.

3.4.3 Iterative Prototyping (Cycle II — Primary)

Cycle II is organised into four implementation increments, each targeting one of the functional areas identified in the Cycle I problem characterisation. The increments address, in sequence, project navigation and file management, editor intelligence, build integration, and debug integration. The scope of each increment is defined at its start as a set of acceptance criteria expressed in terms of the corresponding user stories from the Cycle I problem characterisation. The implementation strategy and concrete design decisions taken within an increment are not predetermined at the start of the cycle; the resulting design decisions are reported in Chapter 4.

Each increment is executed as a complete build-evaluate loop. After implementation, both thesis authors exercise the increment against a representative project before any external session is scheduled, to avoid consuming practitioner time on integration issues that would otherwise have been caught internally. A 20–30 minute informal feedback session with one or two practitioners then exercises the increment on a realistic task; the procedure for these sessions is described in Section 3.5.2. Where increment feedback indicates that a prior design decision may no longer satisfy the increment's acceptance criteria, the relevant Architecture Decision Record [22] is updated or replaced while retaining the earlier rationale for traceability. ADRs are kept short and follow a Context–Decision–Alternatives–Consequences format, so that the chain from problem evidence to design decision to code remains traceable.

3.4.4 Lightweight Architectural Trade-off Analysis (Cycle II — Primary)

The trade-off activity compares candidate integration architectures against explicit quality attributes derived from the Cycle I problem characterisation. The attributes cover project-state synchronisation, editor-support coverage, build-integration completeness, implementation complexity, and the risk of damaging the proprietary project file. The

activity is lightweight rather than a full formal architecture-evaluation method such as ATAM [23]; the comparison and ADR treatment are described in Section 3.6.

3.5 Evaluation Methods (RQ3)

The methods used to address RQ3 are organised around three evaluation touchpoints. Cycle I includes a light usability probe, per Knauss's Guideline G5; Cycle II includes informal practitioner feedback after each implementation increment; and Cycle III conducts the primary evaluation of the artefact through an asynchronous remote evaluation questionnaire combining a six-task self-directed task battery with per-task self-report, the System Usability Scale [24], and a set of reflective comparison questions. The Cycle III design was initially intended to include interview-based evaluation, but participation availability and the remaining project time made an asynchronous questionnaire the practical option. The questionnaire therefore combines several forms of participant-reported evidence: reported task outcomes, a standardised usability rating, and open-ended reflections on the prototype compared with the existing workflow. These sources are collected together as questionnaire-based evidence about task experience, perceived usability, and perceived workflow value.

3.5.1 Initial Usability Probe (Cycle I — Light)

Per Knauss's Guideline G5 [14], Cycle I includes a light usability probe in which the initial extension scaffold is shown to one or two engineers to check whether the problem characterisation remains plausible. The probe is included for methodological completeness only; no retained usability dataset is generated from it, and substantive evaluation evidence comes from Cycle III.

3.5.2 Informal Practitioner Feedback Sessions (Cycle II — Informal)

Cycle II contributes informal RQ3 evidence: short feedback sessions held after each implementation increment, intended to confirm that the increment behaves as expected and to surface practical issues before the next increment is committed. These sessions are not the primary Cycle III evaluation and are not treated as formal usability-test sessions. Each session lasts 20–30 minutes, is held with one or two practitioners, and asks the participant to perform a short realistic task that exercises the increment under review. Three lightweight questions are asked at the end: whether the behaviour matches the participant's expectation, what felt wrong or missing, and whether the participant would be willing to use this in their own work as it stands. A reflective question about the Cycle I pain-point characterisation is also asked, as described in Section 3.3. Responses are used both operationally, to inform the accept-or-revise decision at the end of each increment, and documentarily, to be summarised in the Cycle II feedback notes and referenced in the relevant ADR.

3.5.3 Cycle III Remote Evaluation Questionnaire (Primary)

Cycle III conducts the primary evaluation of the artefact through an asynchronous remote evaluation questionnaire. Participants install the prototype on their own workstation, attempt the supplied task battery against a valid representative PLUS+1 project, and complete the questionnaire in their own time. Consent and data-handling procedures are described in Section 3.7; the questionnaire is reproduced in the appendix.

Background section (Part A). Part A captures two background items used to interpret responses: the participant's self-described current use of VS Code outside the study (item B1), and whether they have used VS Code specifically for C or C++ development before the study (item B2). The identifier scheme P1, P2, ... is allocated by order of submission, consistent with the convention used in Cycle I.

Task battery and per-task self-report (Part B). Six self-directed tasks are issued, each targeting one of the pain-point themes established in the Cycle I thematic analysis. Task 1 (find and fix three syntax errors) targets IntelliSense diagnostics, build-feedback diagnostics, and the Problems-panel path. Task 2 (use Go to Definition to inspect a function's behaviour) targets code-navigation pain points. Task 3 (find all call sites of a symbol) targets project-wide search and Find All References. Task 4 (build the project from inside VS Code without switching to GUIDE) targets the build-trigger path. Task 5 (set a breakpoint and inspect a variable value during a debug session) targets the DAP-based debugger increment. Task 6 (add a new C source file and confirm it appears in the project tree) targets the project-management path that updates the `.plx` project file. A valid representative PLUS+1 project is supplied to participants so that all participants attempt the same task sequence on the same project structure.

For each task, participants complete four self-report items. Item SR1 records the completion outcome on a four-level scale (fully completed; partially; not completed; no answer), with an associated free-text field for partial or unsuccessful completions. Item SR2 records an estimated time-on-task in minutes; this value is treated as a self-estimated rather than a measured quantity. Item SR3 records perceived task difficulty on a five-level scale (very easy; easy; moderate; difficult; very difficult). Item SR4 records the participant's perceived comparison with the GUIDE-only workflow on a five-level scale (faster; about the same; slower; would not have been able; not sure), with an accompanying free-text reasoning prompt. SR4 elicits a within-participant perceived comparison: each participant who has just attempted a task with the extension is asked to estimate how the same task would have unfolded in their normal GUIDE-only workflow, drawing on their own pre-existing experience.

System Usability Scale (Part C, SUS items). The standard ten-item System Usability Scale [24] is administered immediately after the task battery, before the reflective items. The ten statements are presented in the canonical order with five-point Likert response options (strongly disagree to strongly agree). The scoring and interpretation procedure is described in Section 3.6.

Reflective items (Part C, items C1–C8). Eight reflective free-text items follow the SUS and support asynchronous comparison between the prototype and the existing workflow. Items C1–C5 probe the workflow comparison from five angles: overall difference (C1), perceived Cycle I pain-point resolution (C2), unresolved Cycle I pain points (C3), platform-level benefits of working inside VS Code (C4), and adoption intent with stated conditions (C5). Items C6–C8 elicit the participant's top improvement suggestion (C6), features expected but not found (C7), and an unconstrained final comment (C8).

Distribution and consent. The questionnaire is distributed by email to the engineers who are available and willing to take part in the evaluation. As with the Cycle I questionnaire, submission constitutes consent. Because Cycle III asks participants to engage with

the prototype before answering, the consent statement also confirms that the tasks should be attempted before the survey items are completed.

Data handling. Submitted responses are exported from Google Forms to a study spreadsheet and the original form responses are then deleted from Google. The exported data is stored in the access-controlled folder used for all study data.

3.6 Data Analysis

The analysis approach is tailored to the nature of the data collected in each cycle and for each research question.

Requirements Synthesis from Interview and Questionnaire Data (RQ1). Interview transcripts and questionnaire responses are analysed jointly to produce a structured user-story catalogue. The thesis authors read all transcripts and questionnaire responses in full, identify recurring pain points, workarounds, and unmet needs across respondents, and synthesise each recurring concern into a user-story requirement with an associated acceptance criterion, a functional category, a MoSCoW priority, a safety-criticality flag, and the set of respondents who raised it. Synthesis is conducted jointly by the thesis authors in shared review passes rather than independently with formal inter-rater reliability statistics; the implications of this for validity are discussed in Chapter 5. The resulting catalogue is stored as a single spreadsheet with one row per requirement and columns for requirement ID, category, user story, acceptance criteria, MoSCoW priority, safety-critical flag, source respondents, supporting notes (including representative verbatim quotes), and disposition with respect to the Cycle II prototype. Respondents are referred to as P1, P2, ... in the spreadsheet and in the thesis regardless of the collection mode (interview or questionnaire); the catalogue does not preserve a per-row record of which mode produced each contribution.

MoSCoW priorities (Must, Should, Could, Won't) are assigned by the thesis authors in a joint review pass on the synthesised requirements, using two criteria: (i) the frequency with which respondents mentioned the pain point or capability, and (ii) requirements and constraints raised by the industrial supervisors, including dependency relationships between requirements. Participants are not asked to rate priorities directly. Disagreements between the authors are resolved by discussion; where consensus could not be reached, the more conservative priority is recorded. The priorities are thus an analyst judgement informed by participant data and industrial-supervisor requirements, not a participant vote.

A pain point was counted as cited if the respondent affirmatively mentioned it in any section of the interview or questionnaire, without prompting. Absence of mention was not treated as absence of the problem; this counting rule applies to all frequency claims reported in Chapter 4.

Because the interview and questionnaire instruments cover the same five themes, the two datasets are pooled and synthesised in a single pass rather than treated as independent observations of the same phenomenon. The thesis therefore describes this material as pooled across collection modes. Technical documentation is read informally when it is needed to clarify concrete design or implementation questions and is used as a consistency check on the resulting requirements, not as a third coded dataset. The Cycle II reflective feedback question and Cycle III items C1–C3 are reviewed against the Cycle I requirements catalogue to check whether the original pain-point characterisation remains relevant during later cycles.

Architectural Trade-off Analysis (RQ2). The solution space identified through the background review, prototype implementation, and Cycle II walkthrough feedback is examined by comparing each architectural candidate against the five quality attributes defined in Section 3.4.4: project-state synchronisation, editor-support coverage, build-integration completeness, implementation complexity, and risk to the proprietary project file. The final attribute is treated as dominant because a wrapper that damages the file consumed by the certified toolchain has no realistic path to adoption. Candidate architectures are scored on a coarse three-level scale (low, medium, high) and entered into a single trade-off matrix. Walkthrough feedback and implementation notes are summarised, and the architectural decisions taken during implementation are documented as Architecture Decision Records [22]. Each ADR follows a short Context–Decision–Alternatives–Consequences format. Where evaluation feedback during an increment indicates that an earlier decision may no longer satisfy its acceptance criteria, a new ADR is written to revise or supersede the earlier one, while the predecessor is retained so that the revision history remains auditable. The chosen design is then documented with its rationale and known trade-offs in Chapter 4.

Cycle III Questionnaire Analysis (RQ3). The Cycle III questionnaire data is analysed using three complementary procedures aligned with the three classes of items in the instrument.

Per-task self-report data (SR1–SR4) is analysed descriptively. For each of the six tasks, the four-level SR1 completion outcomes are tabulated as counts, and any free-text explanation accompanying partial or unsuccessful completions is coded under the relevant Cycle I pain-point theme. SR2 time estimates are reported per participant rather than aggregated, given their estimated character and the small sample. SR3 perceived-difficulty distributions are tabulated across the five-level scale. The SR4 condition-comparison item is reported as a perception measure rather than a performance measure. SR4 response counts are tabulated per task across the five-level scale, and the accompanying SR4 reasoning text is coded thematically against the Cycle I pain-point themes to identify which extension features participants attribute the perceived difference to.

SUS scores are computed per participant using the standard scoring procedure [24] and reported both individually and as a sample mean, with the adjective interpretation drawn from Bangor et al. [25]. Given the expected small sample, the SUS result is presented as a descriptive indicator of perceived usability rather than as a population estimate. Where free-text responses indicate that prototype features did not execute in a participant's session, the corresponding SUS score is reported but flagged as potentially affected by environment or setup conditions.

Reflective items (C1–C8) are read jointly with the SR4 free-text reasoning and summarised descriptively against the Cycle I requirements catalogue. For each item, the thesis authors group responses by the Cycle I pain-point category they relate to, and report adoption intent (primarily from C5), improvement suggestions (primarily from C6 and C7), and any stated concerns or limitations in the participants' own words. Findings supported by only a single respondent are flagged as indicative rather than conclusive.

Consistency and inconsistency across SUS, SR4 reasoning, and C1–C8 are reported explicitly in Chapter 4. Where multiple questionnaire items support the same interpretation, the finding is presented as better supported within the questionnaire data; where items point in different directions, the inconsistency is reported explicitly rather than reconciled silently.

3.7 Ethical Considerations

All data collection activities involve human participants employed at Danfoss Power Solutions AB. The following requirements apply throughout the study and across all instruments described in Sections 3.3 to 3.5.

Informed Consent. All participants receive the written information sheet reproduced in Appendix A before agreeing to take part. The sheet describes the study purpose, the voluntary nature of participation, and the intended use of the data. Withdrawal at any time, for any reason, has no professional consequence. Two consent mechanisms are used, depending on the instrument. Interview participants and prototype walkthrough participants confirm consent in writing or by email before each session, using the consent form reproduced in Appendix B. Questionnaire participants consent by submission: the consent statement appears at the top of the Google Form, and submission of the completed form confirms that the participant has read the information sheet, understands that participation is voluntary, and consents to anonymised use of their responses.

Anonymisation and Data Minimisation. Only data directly relevant to the research questions is collected. No personally identifying data beyond role title and years of experience is recorded, and no personally identifying questions are asked through the questionnaire. All participants are referred to exclusively by anonymised identifiers (P1, P2, ...) in all data files, analysis documents, and in the thesis itself. Cycle III self-reported task data is presented in aggregate or per-identifier only; no data point is traceable to an individual by a colleague. The questionnaire consent statement makes explicit that responses will not be shared with the participant's employer or manager and that self-reported task outcomes are not used to evaluate participants' professional abilities.

Data Storage and Retention. Raw data, including transcripts, exported questionnaire responses, and retained feedback notes, is stored in an access-controlled cloud folder accessible only to the thesis authors. Questionnaire submissions are exported from Google Forms and the original form responses are then deleted.

Confidentiality of Company Information. Any proprietary information concerning PLUS+1 GUIDE's internal architecture or Danfoss's development processes that is not publicly available is handled in accordance with the non-disclosure agreement between the thesis authors and Danfoss Power Solutions AB. Confidential documentation is not reproduced in the thesis; findings derived from such documentation are reported at an appropriate level of abstraction, with a note that the source is an internal company document. The PLUS+1 project used for Cycle III evaluation does not contain proprietary production data: a valid representative project is used instead.

Summary

The three cycles described in this chapter each address all three research questions, with the balance of attention shifting from problem investigation in Cycle I to solution design in Cycle II to evaluation in Cycle III. Together, the methods combine qualitative problem investigation, iterative prototype construction, lightweight architectural trade-off analysis, and asynchronous artefact evaluation. All activities involving participants adhere to the informed-consent, anonymisation, and data-handling obligations set out in Section 3.7.

4. Results and Analysis

This chapter presents the findings of the study, followed by an analysis and interpretation of the results. The Results section reports the data gathered through the three Design Science Research cycles without interpretation: the problem-investigation evidence from Cycle I (RQ1), the design artefact produced across Cycles I–II (RQ2), and the evaluation evidence from Cycle III (RQ3). The Analysis section then examines patterns across the three datasets and draws conclusions linked back to the research questions.

4.1 Results

The results presented in this section depend on the research methods described in Chapter 3. The study comprises three cycles, each contributing data to a different research question. Cycle I produced a characterisation of the problem space through interviews and a self-completion questionnaire (RQ1). Cycles I–II produced the design artefact: a Visual Studio Code extension implemented as twelve Architectural Decision Records and structured around four functional increments (RQ2). Cycle III produced a remote, task-based evaluation of the prototype with five practitioners (RQ3). Each subsection below reports raw data without interpretation; analysis follows in Section 4.2.

4.1.1 Cycle I – Problem Investigation (RQ1)

Cycle I gathered evidence from ten PLUS+1 GUIDE practitioners through two complementary instruments: two semi-structured interviews and a self-completion questionnaire returned by eight respondents. Background data showed a mean of approximately 11.5 years of GUIDE experience per participant (range 3–25 years), spanning Application Software, Library, Autonomy, and Systems teams, with self-reported C/C++ involvement ranging from ≈10% to ≈50% of working time. All ten participants reported prior use of at least one modern code editor for non-GUIDE work, most commonly Visual Studio Code or Notepad++.

Joint synthesis of the qualitative data, conducted as described in Section 3.6, yielded forty-nine user-story requirements (REQ-01 through REQ-49) grouped into ten functional categories. Each requirement records the originating participants, an acceptance criterion, a MoSCoW priority, and a safety-criticality flag. The full requirements and supporting quote-evidence spreadsheet is reproduced in Appendix E; the distribution of MoSCoW priorities is summarised in Table 4.1.1.

Table 4.1.1: MoSCoW distribution of the forty-nine user-story requirements elicited in Cycle I. Percentages may not sum to 100 due to rounding.

Priority	Count	Share
Must	13	27%
Should	17	35%
Could	19	39%

Five pain points were cited by a majority of respondents. The most frequently mentioned, with nine of ten respondents, was the absence of real-time error highlighting and

IntelliSense-equivalent feedback in the GUIDE C editor (REQ-01, REQ-02). Eight respondents cited the absence of Ctrl+Click navigation to definitions and references (REQ-04, REQ-05). Participants estimated full-project build times ranging from approximately three to twenty minutes depending on project complexity (REQ-41), with one participant (P4) reporting five to ten build invocations per hour; these figures are recalled estimates rather than measured values, reported here as the order of magnitude that practitioners describe. Six respondents reported a workflow that involved keeping a second editor (Notepad++ or VS Code) open on a separate screen and reloading files into GUIDE manually before each build (REQ-14). Five respondents specifically described the multi-step "C Code POU" wrapper procedure as a recurring friction point for introducing C code into a GUIDE project (REQ-21, REQ-39).

Three categories of requirement were flagged as safety-critical: AI assistance (REQ-30, REQ-31, REQ-32), reliability and backward compatibility (REQ-33, REQ-34), and static analysis (REQ-28). One representative quote from each pain point is reproduced verbatim in the same appendix together with its participant identifier.

4.1.2 Cycles I–II – Design Artefact (RQ2)

The design artefact produced across Cycles I and II is a Visual Studio Code extension that connects the PLUS+1 GUIDE embedded IDE with the VS Code editor environment. This section summarises the headline outputs for cross-reference.

A schematic of the extension's runtime structure — showing how the `.p1x` parser feeds the tree view and the file-management module, how stock C/C++ tooling is configured via a static `.clangd` template, how headless builds are launched through the VS Code Task API with attached problem matchers, and how debug sessions are bridged to `cppdbg` — is given as Figure 4.1.1.

The extension is organised as four functional increments, each corresponding to a category of Cycle I requirements: Increment I1 (project navigation and file management) maps to REQ-36, REQ-37, and REQ-38; Increment I2 (editor intelligence) maps to REQ-01 through REQ-05 and REQ-12; Increment I3 (build integration) maps to REQ-15 through REQ-18 and REQ-41; and Increment I4 (debugger integration) maps to REQ-27. The implementation is documented as twelve Architectural Decision Records [22], of which ten carry *Accepted* status and two carry *Superseded* status (ADR-006 superseded by ADR-005, and ADR-008 superseded by ADR-012), reflecting two in-cycle design revisions discussed below.

The four increments together cover approximately half of the forty-nine elicited requirements. The remaining requirements fall into three groups. First, the Could-priority items in the MoSCoW distribution (Table 4.1.1) were placed out of scope for the proof-of-concept prototype and recorded as future work in Section 6.2. Second, the AI-assistance, static-analysis, and reliability requirements flagged as safety-critical were not addressed because the prototype is not intended for certified deployment (Section 5.3). Third, a small number of Should-priority items concerning POU, SCS, and CAN/DBC workflows fall outside the C/C++ editing boundary established in Section 3.1. The full mapping from requirements to increments and dispositions is given in Appendix E.

Editor intelligence in Increment I2 is delivered by configuring stock `clangd` through a workspace-level template, rather than by modifying or extending `clangd` itself and rather than by extracting per-file compilation context from the `.p1x` project format. The extension writes two files to the workspace root on activation and after every successful headless build: a static `.clangd` YAML configuration, and a generic `compile_commands.json`

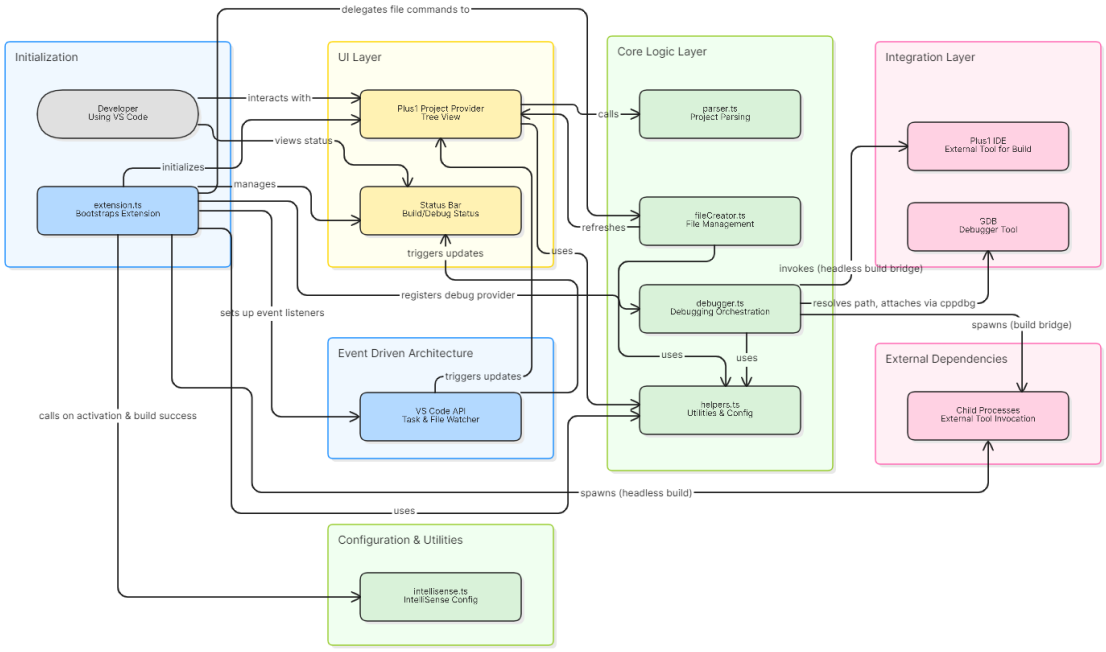


Figure 4.1.1: Runtime architecture of the PLUS+1 GUIDE VS Code extension, organised across four layers. The *Initialization* layer shows `extension.ts` bootstrapping the extension and wiring all modules. The *UI Layer* exposes the project tree view (`Plus1ProjectProvider`) and a build/debug status bar, both managed exclusively by `extension.ts`. The *Core Logic Layer* contains the four functional modules — project parsing, file management, debugging orchestration, and shared utilities — with no cross-module dependencies except through `helpers.ts`. The *Integration Layer* interfaces with external tools: `Plus1_IDE.exe` is invoked for headless compilation via the VS Code Task API and for the debug build bridge via a separate child process; GDB is attached through the Microsoft `cppdbg` debug adapter. An *Event-Driven Architecture* subsystem — comprising VS Code Task and file system watchers — drives UI refresh on build success and `.p1x` file changes; build success additionally triggers IntelliSense regeneration.

enumerating every `.c` file found by `vscode.workspace.findFiles`. The `.clangd` template suppresses toolchain-specific compiler flags and false-positive diagnostics emitted by the PLUS+1 SDK headers, and excludes the build output directory from background indexing. The accompanying `compile_commands.json` enumerates every `.c` file found in the workspace with a generic `clang` entry, providing `clangd` with a compilation database sufficient for project-wide indexing; per-file flags are not extracted from the `.p1x` model, and richer build-derived compilation context is recorded as a deferred improvement in ADR-012. The contribution of Increment I2 is therefore the *configuration* mechanism that supplies a usable language-service context, not extraction of compilation semantics from the proprietary project format.

This delegation choice is the direct application of the design heuristic stated in Section 3.4.4: integration solutions should prefer delegation to existing platform infrastructure over custom re-implementation. The limits of this choice — the absence of automatic TriCore SDK include-path injection, the absence of project-wide diagnostics for unopened files (a clangd protocol property, see Section 6.2), and the inability to address any non-standard-C constructs that might arise from GUIDE-generated code — are treated in Section 5.3.

The architectural design space was reduced to a single trade-off table comparing five architecture candidates (A1–A5) against five quality attributes derived from the Cycle I problem analysis: QA1 Synchronisation Correctness, QA2 IntelliSense Coverage, QA3 Build Integration Completeness, QA4 Implementation Complexity, and QA5 `.p1x` Corruption Risk. The full matrix is reproduced in Table 4.1.2. Candidate A2, a lightweight surgical wrapper around the `.p1x` project file, was selected as the only candidate that combined High build-integration completeness with Low corruption risk and Low–Medium implementation complexity. The matrix is presented as a comparison artefact rather than as the output of a formal architecture-evaluation method; the scoring uses a coarse three-level scale (Low, Medium, High) and was assigned by the thesis authors based on the background review and Cycle I evidence.

Table 4.1.2: Cycle II architecture-candidate trade-off matrix. QA5 is the dominant constraint. A2 (selected, shown in bold) is the only candidate that scores Low on QA5 while also delivering High build integration.

Candidate	**QA1**	**QA2**	**QA3**	**QA4**	**QA5**
A1 Pass-through (open-as-folder)	Low	Low	Low	Low	Low
A2 Lightweight surgical wrapper *	Med	Med	High	Low–Med	Low
A3 `.p1x` round-trip via DOM	High	Med	High	Med	Med
A4 Build-derived compile context	Med	High	High	High	Low
A5 Custom LSP+DAP server	High	High	High	Very high	Low

The extension's internal structure follows a three-layer organisation consistent with the selected A2 wrapper approach. The *presentation layer* consists of the VS Code tree view, status bar, and registered commands, all managed by a single orchestration module (`extension.ts`) that imports every other module but is not imported by any of them. The *application layer* contains the four functional modules — project parsing, file management, build integration, and IntelliSense configuration — each with a narrow, single responsibility and no cross-module knowledge except where explicitly imported. The *infrastructure layer* wraps external tools: `Plus1_IDE.exe` for headless compilation and the build bridge, and GDB via the Microsoft `cppdbg` debug adapter. This separation keeps coupling low, makes each module independently testable, and ensures that changes to the proprietary toolchain interface are localised to the infrastructure layer.

A further consequence of the QA5 corruption-risk constraint is that all `.p1x` mutations in the file-management module are performed through targeted string operations rather than by round-tripping the XML through the project parser. This preserves attribute order, whitespace, and comments in the project file exactly as `Plus1_IDE.exe` wrote them, eliminating any risk of silent reformatting that could cause the certified toolchain to reject the file.

The headless build path uses `CustomExecution` with `cp.spawn` rather than VS Code's `ProcessExecution` because `Plus1_IDE.exe` ignores the `-headless` flag when a pseudo-terminal (PTY) is attached and opens its GUI instead; `ProcessExecution` allocates a PTY, so it cannot be used in this context. This hardware-level constraint was

discovered during Cycle II testing and is an example of proprietary toolchain behaviour that the wrapper must accommodate without any upstream change to the tool.

Internal evaluation throughout Cycle II was conducted as weekly walkthroughs and four informal practitioner feedback sessions, one per increment. The full list of feedback items raised across the sessions, with the disposition (Delivered, Future Work, Out of Scope, or vendor-side resolution) for each item, is given in Appendix F. Feedback drove the two ADR supersessions noted above and also produced a set of concrete deficiencies that were resolved within Cycle II rather than carried forward.

The two supersessions are recorded as follows. ADR-006 (build diagnostics by GCC-regex parsing into a custom `DiagnosticCollection`) was superseded by ADR-005 (headless builds as VS Code Tasks with attached `$gcc` and `$plus1-linker` problem matchers). The supersession deleted approximately 140 lines of custom diagnostic code and replaced them with declarative problem matchers, while preserving the Problems-panel integration that the original design targeted. The trigger was practitioner feedback that build output was visible only at task completion under the earlier `cp.spawn`-based design; the Task-API path streams output to the Problems panel as the terminal output flows.

ADR-008 (Makefile-scraping pipeline that wrote `c_cpp_properties.json` for the `ms-vscode.cpptools` extension) was superseded by ADR-012 (static `.clangd` at the workspace root, plus a generic `compile_commands.json`). The supersession had two technical triggers: the initial `.clangd` write location, inside `.vscode/`, was silently ignored by `clangd` because its configuration search walks up the directory tree from each source file and `.vscode/.clangd` is not on that path; and the Makefile only existed after a successful build, making IntelliSense unavailable on first workspace open. The superseding design writes `.clangd` to the workspace root and supplies the information `clangd` needs without depending on a prior build, satisfying the immediate-availability requirement raised in the walkthrough feedback.

4.1.3 Cycle III – Evaluation (RQ3)

Cycle III evaluated the prototype with five practitioners through a task-based remote questionnaire delivered between 30 April and 9 May 2026. Each participant attempted six representative tasks using the extension on a prepared sample project, then completed the System Usability Scale [24] and an eight-item structured comparison against their normal GUIDE workflow. Participants are referred to as P1–P5 in the order their responses were received. All five confirmed that they had previously participated in Cycle I. Because Cycle I responses were anonymised and not keyed to identifiable individuals, the Cycle III identifiers cannot be linked back to specific Cycle I responses, and it is not known whether a given Cycle III respondent had previously contributed via the interview or via the questionnaire. The cohort-level continuity between the two cycles is therefore real, but the comparison in Section 4.2.4 is aggregate-to-aggregate rather than within-subject.

4.1.3.1 Task Outcomes

The six tasks correspond to the four implementation increments and together exercise the full edit–build–debug round-trip. Task 1 (diagnose three syntax errors using IntelliSense feedback, the Problems panel, and Find in Files) exercised editor intelligence and build diagnostics; the three intended faults were a missing semicolon, a missing bracket, and an incorrect variable reference. Task 2 (use *Go to Definition* to inspect a function's return behaviour) and Task 3 (use *Find All References* to enumerate call sites) exercised

IntelliSense navigation; Task 4 (build the project from VS Code) exercised the Task API integration; Task 5 (set a breakpoint and read a variable value at the breakpoint hit) exercised the debugger bridge; Task 6 (add a new C file to the project from VS Code) exercised the file-management and `.p1x` synchronisation path.

Self-reported task completion (Yes fully / Partially / No) and self-reported difficulty (Very easy / Easy / Moderate / Difficult / Very difficult) are summarised in Table 4.1.3. Self-reported time-on-task ranged from under one minute to thirty minutes; the wide range on Task 1 is attributable to one participant (P5) reporting that the Problems panel displayed errors only for currently open files, a behaviour also flagged by P1, and one participant (P4) reporting setup-related compiler errors that obscured the three intended errors.

Table 4.1.3: Cycle III task outcomes across five participants (P1–P5). Completion: Y=Yes fully, P=Partially, N=No. Difficulty: VE=Very easy, E=Easy, M=Moderate, D=Difficult, VD=Very difficult.

Participant	**T1**	**T2**	**T3**	**T4**	**T5**	**T6**
P1	Y/VD	N/D	P/D	Y/E	Y/E	Y/E
P2	Y/VE	Y/VE	Y/VE	Y/VE	Y/VE	Y/VE
P3	Y/E	Y/VE	Y/E	Y/E	Y/E	Y/VE
P4	Y/M	Y/E	Y/VE	Y/E	Y/E	Y/VE
P5	N/VD	Y/VE	N/E	Y/E	Y/VE	Y/E
Yes-fully rate	3/5	4/5	3/5	5/5	5/5	5/5

In a structured comparison item (SR4), participants were asked whether each task would have been faster, about the same, or slower in PLUS+1 GUIDE alone. The distribution of responses is given in Table 4.1.4. The two "Faster in GUIDE" responses on Task 1 came from P1 and P5, the same two participants who reported incomplete or partial completion on that task.

Table 4.1.4: Self-reported comparison against PLUS+1 GUIDE alone (Cycle III item SR4), counts out of five participants.

Task	**Faster in GUIDE**	**Same**	**Slower in GUIDE**	**Not possible in GUIDE**
T1 Diagnose errors	2	1	2	0
T2 Go to Definition	0	2	3	0
T3 Find All References	1	0	4	0
T4 Build	0	5	0	0
T5 Debug	0	1	3	1
T6 Add C file	0	2	3	0

4.1.3.2 System Usability Scale

The ten SUS items were scored using the standard procedure of [24]. Individual scores were P1=35.0, P2=95.0, P3=77.5, P4=82.5, P5=77.5 on the 0–100 scale, with a mean of 73.5, median of 77.5, and standard deviation of 22.7. On the adjective-rating scale of [25], the median falls between "Good" and "Excellent", and four of five scores fall above the "Acceptable" threshold of 70. The lowest score (P1, 35.0) corresponds to the

participant who reported in free-text responses that *Go to Definition*, *Find All References*, and the build command did not respond as expected in their session, which is consistent with an installation or environment defect rather than a design issue. The primary reported figure is the full-sample mean of 73.5; the sensitivity of this figure to P1 is noted (excluding P1, the mean is 83.1) but no reclassification of the adjective band is drawn from the four-participant subset. Because the sample is small ($n = 5$), the SUS result is treated throughout as a descriptive indicator, not as a population estimate. Item-level SUS results are given in Appendix G.

A bootstrap 95% confidence interval (10,000 resamples) on the mean spans [53.0, 88.0], showing that with $n = 5$ the mean alone carries little information and that the SUS result must be read descriptively rather than inferentially (seed = 42).

4.1.3.3 Structured Comparison and Adoption Intent

In the structured comparison section (items C1–C8), four of five participants identified the prototype as a clear workflow improvement over their normal GUIDE workflow, citing specifically code navigation, IntelliSense, integrated build, and the removal of the edit-in-Notepad++/reload-in-GUIDE cycle. P1's response was neutral and attributed the limited benefit to scope: "it cannot work with POUs, CCPs, SDLs, HWDs." Four of five participants stated they would adopt the extension in their regular workflow, conditional on the named limitations being addressed. Adoption-blocker conditions stated by respondents included: C++ support and Linux-controller targets (P5), POU interface editing in VS Code (P2), faster build and debug start-up (P3), and full IntelliSense compliance (P4). Verbatim responses to the eight comparison items are reproduced in Appendix H.

4.2 Analysis

This section interprets the findings, identifies patterns across the three datasets, and assesses their significance with respect to the three research questions.

4.2.1 RQ1 – Productivity Limitations

The Cycle I evidence supports four observations.

- **Code intelligence dominates the pain-point distribution.** The five most frequently cited pain points (real-time diagnostics, navigation, find-in-files, file synchronisation, and build latency) account for 18 of the 49 elicited requirements and were named by 8–9 of 10 respondents. This is consistent with the position of Beningo [2] that embedded toolchains have under-invested in editor-side capabilities relative to mainstream development environments.

- **Friction is multiplied by build latency.** Self-reported full build times of 3–20 minutes and self-reported build frequencies of 5–10 per hour together suggest that errors not caught at edit time impose a multi-minute round-trip cost. These figures are recalled estimates rather than measured values and are reported here only as the order of magnitude practitioners describe; even at the low end of both ranges the round-trip cost is large enough to make the absence of real-time diagnostics a productivity issue and not only a usability one, which is consistent with its placement as the top-cited pain point and the primary driver behind REQ-01.

- **Workarounds are widespread and structurally similar.** Six of ten respondents reported the same workaround: a second editor (Notepad++ or VS Code) running on a separate screen, with manual save-and-reload into GUIDE before each build. The uniformity of this workaround suggests an editor-side "shadow IDE" had already emerged informally, and is one of the clearest indicators that a formal integration would address existing demand rather than create new demand.

- **Safety concerns concentrate around AI and reliability, not core editing.** Only 9 of 49 requirements were flagged as safety-critical, and these cluster in the AI-assistance, reliability, and static-analysis categories. The core editing and navigation requirements (REQ-01 through REQ-08) were flagged Must or Should but were not marked safety-critical, which establishes a defensible scope for a Cycle II prototype that targets the editor-side experience first and defers AI-mediated workflows.

4.2.2 RQ2 – Solution Design

The Cycle II artefact translates the four most frequently cited Cycle I pain points into four corresponding increments, with architectural decisions documented as twelve ADRs. The trade-off analysis selected candidate A2 (a lightweight surgical wrapper) over the alternatives on the basis that QA5 (`.p1x` corruption risk) acts as a dominant constraint: the `.p1x` file is the canonical project file consumed by the certified GUIDE toolchain, and any architecture that normalises or re-serialises it on every write introduces silent corruption risk incompatible with the safety-critical context identified in Section 4.2.1.

The two in-cycle ADR supersessions (ADR-006 $\rightarrow$ ADR-005 for build diagnostics, and ADR-008 $\rightarrow$ ADR-012 for IntelliSense) illustrate the regulative cycle operating at the intra-increment level. In both cases the supersession was triggered by evaluation feedback within the same increment, not by external requirements changes; both transitions also reduced implementation complexity by delegating work to existing VS Code or `clangd` infrastructure (problem matchers; the `.clangd` config-search walk), which is consistent with the design heuristic stated in Section 3.4.4 that integration solutions should prefer delegation to the platform over custom re-implementation.

Candidates A4 and A5 were not carried forward: A4 would couple the extension more tightly to the `.p1x` schema without a proportional reduction in corruption risk, and A5's engineering scope would shift the project from an incremental wrapper to a full custom language server, contradicting the delegation heuristic of Section 3.4.4.

4.2.3 RQ3 – Evaluation

The Cycle III data supports three observations.

The structured-comparison item SR4 (Table 4.1.4) is reported throughout this section as a within-participant perception measure rather than as observed performance, as discussed in Chapter 5.

- **Headline usability sits in the "Good" adjective band, with a wide spread.** The full-sample SUS mean of 73.5 places the artefact in the "Good" band [25], and the median (77.5) sits at the boundary with the "Excellent" band. The standard deviation of 22.7 across five participants is large and is driven principally by P1's score of 35.0, whose free-text responses indicate that three of the six tasks were unresponsive in their session, pointing to an installation or environment defect rather

than a design defect. The sensitivity of the mean to this single observation is noted but no stronger claim about the adjective band is drawn from the four-participant subset, given the descriptive status of SUS at $n = 5$.

- **Task outcomes match the design intent of each increment.** The 100% completion rates on Tasks 4 (build), T5 (debug), and T6 (add C file) correspond to the Cycle II increments I3, I4, and I1 respectively, which were designed against acceptance criteria expressed directly in terms of these task types. The lower completion rates on Tasks 1 and 3 reflect specific deficiencies that were surfaced in the Cycle III responses: the Problems-panel-only-for-open-files behaviour on Task 1, and an intermittent context-menu failure on Task 3 reported by P1. Both are concrete enough to be carried directly into the future-work list (Section 6.2).

- **Adoption intent is conditional and bounded.** Four of five participants indicated they would adopt the extension, but each named a specific blocking condition (C++ support, POU interface editing, faster build start-up, full IntelliSense compliance). The bounded nature of these conditions is informative: no participant indicated that the integration approach itself was unsuitable, and the conditions cluster around scope expansion (C++, POU, Linux targets) rather than rearchitecture. P1's neutral position is the one exception and is explicitly attributed to scope ("it cannot work with POUs, CCPs, SDLs, HWDs"), which agrees with the other participants on the same axis.

A statistical significance test on the SUS scores or on task completion rates was not conducted: with $n = 5$ Cycle III participants the test power would be inadequate to detect anything below very large effect sizes, and no comparison baseline (e.g. a parallel cohort using GUIDE alone) was constructed. The Cycle III data is therefore reported as evaluation evidence in the Design Science Research sense, not as the result of a controlled experiment.

4.2.4 Consistency Between Cycle I and Cycle III Responses

A useful internal check is whether the Cycle III responses are consistent with the Cycle I findings at the cohort level. Because Cycle I responses were anonymised, individual Cycle III responses cannot be paired with individual Cycle I responses; the comparison is therefore aggregate-to-aggregate. Two correspondences are observable. First, the Cycle III SR4 responses (Table 4.1.4) record "slower in GUIDE" as the modal response for navigation tasks (T2, T3) and add-file (T6), and "about the same" for build (T4), which is consistent with the Cycle I pain-point ranking that placed code intelligence and `.plx`/editor synchronisation among the most frequently cited friction points. Second, items raised but deferred during the Cycle II informal feedback sessions (build slowness, debug start-up time; see Appendix F) reappear in Cycle III under items C3 and C6, indicating that unresolved Cycle II issues remained visible at evaluation time. No individual-level claim is made about whether participants who expressed adoption intent in Cycle III correspond to particular Cycle I priority patterns, because Cycle I anonymity prevents that form of linking and because MoSCoW priorities in this study are analyst-assigned (Section 3.6) rather than participant-ranked.

These results carry forward into Chapter 5, where their significance is interpreted in the context of the literature reviewed in Chapter 2.

5. Discussion

In this chapter we reflect on the findings reported in Chapter 4 and assess whether the research problem stated in Chapter 1 has been effectively addressed. The project is structured as a feedback loop: a problem was defined (RQ1), an appropriate methodology was selected (Design Science Research, Chapter 3), a design artefact was constructed across three cycles (RQ2), and evaluation data was gathered (RQ3). The analysis of this data is now interpreted with respect to its significance, its position within the existing literature, and the limitations that constrain how far the findings can be generalised.

5.1 Significance of the Findings

The findings most directly relevant to the research questions cluster around two observations.

First, the Cycle I pain-point distribution (Chapter 4 Section 4.2.1) gives empirical weight to a claim that has previously been made in general terms about embedded toolchains [1, 2]: that the productivity gap is concentrated in the editor layer (navigation, real-time diagnostics, file synchronisation) rather than in the build, simulation, or target-deployment layers, which are generally well served by the existing certified toolchain. The practical implication is that an integration layer focused on the editor layer may capture a substantial part of the observed productivity gain without disturbing the certified portion of the toolchain.

Second, the SUS result (mean 73.5, median 77.5) places the artefact in the "Good" adjective band of Bangor et al. [25], with the median sitting near the boundary with "Excellent", despite a wide standard deviation driven primarily by one participant with a likely environment defect. The significance is not that the absolute score is high, since SUS at $n = 5$ is best treated as descriptive rather than inferential, but that the score is achieved by a prototype whose primary contribution sits at the project-format integration layer. The Cycle II artefact delegates the majority of its editing-experience functionality to existing VS Code, `clangd`, and `cppdbg` infrastructure rather than to custom implementation: C source files developed inside the host environment are standard C and are adequately served by `clangd` once a workspace-level configuration template is supplied. The Cycle III data provides weak but consistent evidence that this delegation-oriented approach is acceptable to practitioners in this case.

The findings also produced something that was not expected at the start of the work. The Cycle I evidence showed that practitioners had already, informally, constructed an editor-side "shadow IDE" workflow involving Notepad++ or VS Code as a second editor with manual save-and-reload into GUIDE. Six of ten respondents reported this workaround. This indicates that the demand addressed by the Cycle II artefact was not latent but already visible in working practice, and that the artefact's value is at least partly to formalise an existing workaround rather than to introduce an unfamiliar pattern of work. This framing was not initially anticipated and changed the way the artefact is presented in Chapter 4.

5.2 Comparison with the Literature

Three points of contact with the related-work literature are particularly relevant.

Editor integration for non-mainstream languages. Previous editor-integration work has focused on formal specification languages such as VDM [7, 8] or on graphical modelling languages [18]. The present work addresses a different kind of target: a proprietary, XML-based embedded project format. The contribution sits at the project-format integration layer rather than at the language-service layer, which closes a gap that the prior editor-integration literature does not directly address.

Industrial IDE usability. The empirical baseline most directly comparable to this study is the 4diac IDE usability work of Wiesmayr et al. [17], which used SUS and structured comparison instruments in a similar industrial setting. Wiesmayr et al. reported SUS scores in a comparable range for a different industrial visual-programming IDE. The agreement on instrument and on order of magnitude supports the methodological choice adopted in Chapter 3 and reduces the risk that the present SUS result is an artefact of instrument selection.

Editor and toolchain technical debt. Besker et al. [10] showed that toolchain-related technical debt is consistently underestimated by practitioners and managers and is linked to measurable productivity loss. The Cycle I evidence in the present study is consistent with that account: respondents named specific friction sources (multi-step C-POU wrapping, manual file reload, slow find-in-files) but did not initially frame them as technical debt. The Cycle III adoption-intent data, in which four of five participants would adopt the prototype with bounded conditions, suggests that a working artefact may help practitioners reframe these frictions as addressable debt rather than as fixed properties of the environment.

5.3 Limitations

The following limitations are most relevant to the findings reported above.

Sample size and selection. The Cycle I evidence base of ten participants and the Cycle III evaluation cohort of five participants are small in absolute terms. Both samples were drawn from a single industrial partner (Danfoss Power Solutions AB) and were self-selected from a recruitment notice circulated internally. Nine of ten Cycle I respondents reported prior use of VS Code or Notepad++, which means the requirements catalogue produced in Cycle I represents the pain points of practitioners who already lean toward modern editor tooling, not necessarily of the wider GUIDE user population. This biases the Cycle III adoption-intent result upward on at least two axes: respondents are already familiar with the host editor, and they are more likely to value the kind of capabilities the prototype delivers. The findings should therefore be read as evidence of the pain points and adoption attitudes of the modern-editor-oriented subset of GUIDE practitioners, not as a characterisation of the population as a whole.

Evaluation instrument. The Cycle III instrument used the System Usability Scale [24] together with a set of structured comparison items. SUS at $n = 5$ is descriptive rather than inferential, and the standard deviation of 22.7 across the five scores reflects this constraint. The evaluation was conducted as an asynchronous remote questionnaire because participant availability and the remaining project time did not permit interview-based sessions. As a result, the instrument provides no directly observed task performance:

completion outcomes and task times rely on participant self-report, and time values are estimates rather than measurements. No counterfactual baseline (a parallel cohort using GUIDE alone on the same tasks) was constructed, so the structured-comparison items (SR4) rely on perceived comparison with participants' existing GUIDE experience rather than on observed paired performance. No concurrent verbalisation data was collected, so in-the-moment cognitive friction is captured only where participants chose to describe it in free-text fields. This is appropriate for an evaluation positioned within Design Science Research but limits how strongly the comparative results (Table 4.1.4) can be read as performance claims.

Artefact scope. The Cycle II artefact covers only the C/C++ editing path. POU interface editing, SCS graphical editing, hardware-configuration editing, and CAN/DBC management remain inside the GUIDE environment. Two of the four conditional-adoption responses (P2, P3) explicitly named one or more of these as adoption gates. The findings should therefore be read as evidence for one specific integration boundary, not as a claim about the limits of the integration approach more broadly.

Toolchain version coupling. The Cycle II implementation was developed and evaluated against the PLUS+1 GUIDE 2026.1 snapshot provided for this study, which included the command-line debug support needed for Increment I4. Because this snapshot was the only GUIDE installation used during development and evaluation, behaviour with other GUIDE installations has not been validated.

Reliance on a stock C/C++ language server. The IntelliSense path in Increment I2 relies on the existing `clangd` language server, used unmodified. The prototype therefore inherits `clangd`'s coverage of and limitations for standard C and C++, and does not address any PLUS+1- or GUIDE-specific constructs that fall outside standard C semantics. In particular, any code generated by GUIDE from POUs, SCS diagrams, or hardware-configuration files is treated by the prototype as ordinary C source, with no language-service-level awareness of its provenance or of any non-standard idioms it may contain. During development, an attempt was made to improve symbol resolution by adding generated include paths and toolchain-related flags to the `clangd` configuration, but this did not improve IntelliSense behaviour in practice. It was not established whether the remaining gaps were caused by unsupported constructs in `clangd`, by incomplete or incorrect configuration flags, or by properties of the generated code itself. Whether such constructs exist in practice and warrant language-server-level support, for example through a `clangd` plugin or a thin custom layer, was not investigated in this thesis. The IntelliSense gaps reported by P4 in the Cycle III evaluation are consistent with this limitation but cannot be attributed to it conclusively without further investigation. This question is recorded as future work in Section 6.2.

Cycle II evaluation depth. The Cycle II informal feedback sessions described in Section 3.5.2 were not analysed as formal usability data. Each session was 20–30 minutes long, was held with one or two practitioners, and recorded responses to three lightweight questions rather than a standardised instrument; the output was used operationally to inform the accept-or-revise decision at the end of each increment and documentarily to be summarised in the ADRs. The two supersessions reported in Section 4.2.2 therefore rest on a small number of feedback events rather than on systematic evaluation. This is by

design: Cycle II's primary contribution is to RQ2, not RQ3, and the Cycle III instrument is where the formal evaluation evidence lives. But it bounds how strongly the intra-cycle observation in Section 5.1 can be read: the evidence is sufficient to show that the mechanism worked in this case, not to estimate how often the same mechanism would correct decisions in a larger sample.

Trade-off analysis is informal. The Cycle II trade-off matrix (Table 4.1.2) is a comparison artefact rather than the output of a formal architecture-evaluation method. The scoring uses a coarse three-level scale and was assigned by the thesis authors. No formal sensitivity-point or trade-off-point analysis [23] was conducted, and no external architect reviewed the scoring. The matrix is presented as a transparent record of which alternatives were considered and why A2 was chosen; it does not claim the rigour of a full ATAM exercise.

5.3.1 Validity Threats

This section maps the limitations described above to the standard threats-to-validity framework used in empirical software engineering [26].

Construct validity. The main threat is the reliance on self-reported comparisons (SR4) rather than observed paired performance in two conditions. The instrument asks participants to draw on memory of their normal GUIDE workflow, which is a weaker measure than a controlled within-subjects experiment, and the comparison condition is reconstructed rather than executed. SUS and the open-ended reflective items (C1–C8) provide partial cross-checks within the same instrument and the same cohort, but not independent corroboration; the construct threat is therefore acknowledged rather than mitigated.

Internal validity. The evaluation does not establish a causal effect of the prototype in the way that a controlled comparison would. Participants were self-selected, the sample was small, and the study relied on a prototype walkthrough and retrospective comparison with the normal GUIDE workflow rather than on observed performance in two matched conditions. Positive responses may therefore reflect, in part, novelty, participant expectations, or the attention associated with taking part in the study. The ADR supersessions in Cycle II show that design decisions were revised in response to stakeholder requirements, implementation constraints, and feedback from the industrial supervisor. However, they should not be interpreted as independent evidence of effectiveness, because the researchers still selected the final implementation approach and documented the architectural rationale. To make this process more transparent, the superseded ADRs were retained and the reasons for replacement were linked to concrete technical observations, including the unsuitable `.vscode/.clangd` write location for `clangd` configuration discovery and the observable difference between post-completion problem population and matcher-based problem reporting. These records support traceability from requirements and feedback to design changes, but they do not remove the internal validity threat that researcher judgement influenced how competing implementation options were assessed and prioritised.

External validity. The study is a single industrial case with a single toolchain version. The transferability claims are therefore limited to the design heuristic (delegate to the

platform where possible, treat the proprietary project file as a dominant constraint) rather than to the specific integration artefact.

Conclusion validity. With $n = 5$, SUS scores are descriptive only. The reported mean and median should not be interpreted as population estimates, and the wide bootstrap interval reported in Section 4.1.3.2 reinforces that the adjective-band interpretation must be used with caution. The consistency of the qualitative signals (adoption intent, pain-point resolution) across instruments lends credibility, but no formal statistical test was appropriate.

6. Conclusion and Future Work

This chapter summarises the main findings, states how strongly they can be generalised, and identifies future work raised by the study.

6.1 Conclusion

What was achieved. This thesis investigated the productivity barrier faced by embedded software engineers who develop C/C++ code inside proprietary, domain-specific development environments, using PLUS+1 GUIDE as the industrial case. The work contributes three outputs. First, an empirically grounded characterisation of practitioner pain points from ten PLUS+1 GUIDE users, expressed as forty-nine prioritised user-story requirements (RQ1). Second, a working Visual Studio Code extension, organised as four functional increments and documented as twelve Architectural Decision Records, that connects the PLUS+1 toolchain with VS Code's editor, build, and debug infrastructure (RQ2). The architectural choice between five candidate architectures was made through an informal trade-off analysis in which the corruption risk of the proprietary `.p1x` project file acted as a dominant constraint. Third, a remote, task-based evaluation of the prototype with five practitioners, which produced a mean SUS score of 73.5 (median 77.5) and conditional adoption intent from four of five participants (RQ3).

Why the results matter. For the industrial partner, the work delivers a working prototype together with a prioritised requirements catalogue and a documented set of architecture decisions that can be carried into further development. For practitioners in adjacent embedded toolchains, the findings provide a concrete but single-case example of connecting a proprietary embedded IDE with a modern editor. For the research community, the work contributes an empirically grounded case study of project-format-to-editor-tooling integration in a safety-critical industrial setting, complementing the existing editor-integration literature on formal specification languages and graphical modelling languages. The combination of features studied here — a proprietary embedded project file, integration via stock C/C++ tooling, and a safety-critical industrial context — does not, to the authors' knowledge, appear jointly in the published literature surveyed in Chapter 2; a stronger uniqueness claim would require a systematic literature review beyond the scope of this thesis.

How generalisable the results are. The Cycle I and Cycle III evidence is drawn from a single industrial partner and a modest sample. The findings are therefore presented as evidence from a specific industrial case rather than as claims about embedded toolchain integration in general. The transferability of one specific element is nonetheless plausible: the design heuristic itself (prefer delegation to existing platform infrastructure over custom re-implementation, and treat the proprietary project file as a dominant constraint) is mechanism-level rather than instance-level and should apply wherever a certified toolchain coexists with a modern editor ecosystem. The principal limitations of the study (small samples from a single industrial partner, absence of a counterfactual baseline, scope restricted to C/C++ editing, validation against a single toolchain version) are treated in detail in Section 5.3 and bound how strongly the results can be interpreted without affecting the internal consistency of the three contributions.

6.2 Future Work

The future-work items identified during this study fall into three groups: architectural improvements that were considered during the Cycle II trade-off analysis and deferred for cost reasons; feature requests from the Cycle I and Cycle II feedback sessions that remained out of scope for the proof-of-concept prototype; and concrete deficiencies surfaced in the Cycle III evaluation. The full Cycle I and Cycle II informal-feedback table, with the disposition for each item (*Delivered*, *Future Work*, *Out of Scope*), is given in Appendix F; only items marked *Future Work* are listed below.

Deferred architectural improvements. One of the five candidate architectures considered in Section 4.1.2 was recorded as future work because it directly addresses a limitation observed in the Cycle III evaluation.

- **Build-derived compile context (candidate A4).** A future refinement could derive per-file flags, include paths, and macros from GUIDE's generated build artefacts or documented toolchain configuration, while continuing to use the `.plx` XML only for project metadata and source-file membership. This information could then improve the existing generic `compile_commands.json` and `.clangd` setup. This would target the IntelliSense-coverage gap reported by Cycle III participants P3 and P4, but at the cost of higher implementation complexity (QA4: High) and tighter coupling to GUIDE's build outputs and toolchain assumptions.

Deferred feature work from informal feedback. Several items raised in the Cycle I and Cycle II informal feedback sessions were placed out of scope for the proof-of-concept prototype and are carried forward as future work. The Cycle II items already *Delivered* (project-tree file-type filtering, `.sys` and `.lhx` protection, automatic tree refresh after compile, automatic IDE-path detection, problem-matcher-driven Problems-panel streaming, and the GDB-path resolution on first debug launch) are described as Cycle II results in Section 4.1.2 and are not repeated here.

- **Diagnostic-parameter grid view.** Cycle I feedback requested a user-friendly grid view in VS Code for adding and removing PLUS+1 diagnostic variables (Checkpoint, Set Value, Set Pulse, Retain), with the corresponding XML file updated to reflect changes. The implementation would extend the file-management mechanism delivered in Increment I1 with a structured editor for the diagnostic-variable XML elements.
- **Project creation from VS Code.** Cycle I feedback requested the ability to create a new PLUS+1 GUIDE project from within VS Code, replacing the current GUIDE pop-up dialog with a VS Code command that invokes the documented `Plus1_IDE -create-project` CLI. A single hardware target would be sufficient for the proof-of-concept follow-on; broader hardware-target selection is a longer-horizon item.

Deficiencies surfaced in Cycle III evaluation. Two concrete deficiencies surfaced in the Cycle III responses (Section 4.1.3). Both are scoped at the level of a single implementation change rather than an architectural revision.

- **Problems-panel scope.** Diagnostics produced by the `clangd` configuration in Increment I2 populate the Problems panel only for files currently open in the editor, which contradicts the expectation surfaced in the Cycle III responses for Task 1 that all problems in the project should be visible. This is a property of `clangd` itself rather than of the extension: the LSP `textDocument/publishDiagnostics` message is per-document, and `clangd` only publishes for files in the active document set. Closing this gap would be better addressed within the present wrapper architecture by an extension-side aggregator that performs a post-build diagnostic scan.

- **Context-menu reliability.** An intermittent context-menu failure on Task 3 was reported by participant P1. The behaviour was not reproducible in author testing on the same prototype version. The likely source is the asynchronous tree-refresh path: when `provider.refresh()` clears the in-memory project tree before the re-parse resolves, a context-menu invocation in that window can return menu items computed from a stale node. The follow-on action is to add an explicit pending-parse flag and gate context-menu resolution on it.

Longer-horizon directions. Two further directions are recorded as worth investigating but lie beyond what the present trade-off analysis can scope.

- **Scope expansion beyond C/C++.** Cycle III participants named POU, CCP, SDL, and HWD as the formats not currently handled by the prototype. Whether these are addressable through extension of the present A2 architecture or whether they require a different integration pattern (e.g. a graphical-modelling host inside VS Code) is an open question, not a known cost.

- **Extension-security hardening.** Chapter 2 reviewed recent literature on VS Code extension security [12, 13]. For production deployment in the case organisation, a security review covering cross-extension data exposure, untrusted-input handling at the `.plx` parser, and the privilege profile of the bundled `ms-vscode.cpptools` dependency is required.

Bibliography

[1] P. Liggesmeyer and M. Trapp, "Trends in embedded software engineering," *IEEE Software*, vol. 26, no. 3, pp. 19–25, May-June 2009.

[2] J. Beningo, *Embedded Software Design*. Apress, 2022.

[3] Microsoft, "Visual studio code documentation," 2026, accessed: 2026-05-04. [Online]. Available: https://code.visualstudio.com/docs

[4] D. Bork and P. Langer, "Catchword: Language server protocol: An introduction to the protocol, its use, and adoption for web modeling tools," *Enterprise Modelling and Information Systems Architectures (EMISAJ)*, vol. 18, no. 9, pp. 1–19, 2023.

[5] H. Bünder, "Decoupling language and editor - the impact of the language server protocol on textual domain-specific languages," in *Proceedings of the 7th International Conference on Model-Driven Engineering and Software Development (MODELSWARD 2019)*, Prague, Czech Republic, 2019, pp. 131–142.

[6] Danfoss Power Solutions, *PLUS+1 GUIDE User Manual*, 2026, accessed: 2026-05-22. [Online]. Available: https://assets.danfoss.com/documents/latest/583380/AQ152886483724en-003301.pdf

[7] J. K. Rask, F. P. Madsen, N. Battle, H. D. Macedo, and P. G. Larsen, "Visual studio code vdm support," in *Proceedings of the 18th Overture Workshop*, 2021, pp. 35–50.

[8] ——, "The specification language server protocol: A proposal for standardised lsp extensions," in *Formal Integrated Development Environment 2021 (F-IDE 2021)*, ser. EPTCS, vol. 338, 2021, pp. 3–18.

[9] J. Cederbladh, R. Eramo, V. Muttillo, and P. E. Strandberg, "Experiences and challenges from developing cyber-physical systems in industry-academia collaboration," *Software: Practice and Experience*, vol. 54, no. 6, pp. 1193–1212, Jan. 2024.

[10] T. Besker, A. Martini, and J. Bosch, "Software developer productivity loss due to technical debt—a replication and extension study examining developers' development work," *Journal of Systems and Software*, vol. 156, pp. 41–61, Oct. 2019.

[11] D. Barros, S. Peldszus, W. K. G. Assunção, and T. Berger, "Editing support for software languages: Implementation practices in language server protocols," in *Proceedings of the ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems (MODELS '22)*, Montreal, QC, Canada, Oct. 2022, pp. 232–243.

[12] S. Edirimannage, C. Elvitigala, A. K. K. Don, W. Daluwatta, P. Wijesekara, and I. Khalil, "Developers are victims too: A comprehensive analysis of the vs code extension ecosystem," 2024, arXiv preprint arXiv:2411.07479v1. [Online]. Available: https://arxiv.org/abs/2411.07479

[13] Y. Liu, C. Tantithamthavorn, and L. Li, "Protect your secrets: Understanding and measuring data exposure in vscode extensions," in *Proceedings of the 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2025.

[14] E. Knauss, “Constructive master’s thesis work in industry: Guidelines for applying design science research,” in *Proc. IEEE/ACM 43rd Int. Conf. on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, Madrid, Spain, 2021, pp. 110–121.

[15] S. Nuratch, “A serial bootloader with ide extension tools design and implementation technique based on rapid embedded firmware development for developers,” in *Proceedings of the 2017 12th IEEE Conference on Industrial Electronics and Applications (ICIEA)*, Siem Reap, Cambodia, 2017, pp. 1865–1869.

[16] PlatformIO, “Platformio documentation,” 2026, accessed: 2026-05-15. [Online]. Available: https://docs.platformio.org/en/stable/

[17] B. Wiesmayr, A. Zoitl, and R. Rabiser, “Assessing the usefulness of a visual programming ide for large-scale automation software,” *Software and Systems Modeling*, vol. 22, pp. 1619–1643, 2023.

[18] R. Rodriguez-Echeverria, J. L. C. Izquierdo, M. Wimmer, and J. Cabot, “Towards a language server protocol infrastructure for graphical modeling,” in *Proceedings of the 21th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, Oct. 2018, pp. 370–380.

[19] A. R. Hevner, “A three cycle view of design science research,” *Scandinavian Journal of Information Systems*, vol. 19, no. 2, pp. 87–92, 2007.

[20] R. K. Yin, *Case Study Research and Applications: Design and Methods*, 6th ed. Thousand Oaks, CA: SAGE Publications, 2018.

[21] C. Robson and K. McCartan, *Real World Research*, 4th ed. Chichester: Wiley, 2016.

[22] M. Nygard, “Documenting architecture decisions,” https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions, 2011, accessed: 2026-05-05.

[23] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, and J. Carriere, “The architecture tradeoff analysis method,” in *Proceedings of the Fourth IEEE International Conference on Engineering of Complex Computer Systems (ICECCS)*. IEEE, 1998, pp. 68–78.

[24] J. Brooke, “SUS: A “quick and dirty” usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and A. L. McClelland, Eds. London: Taylor and Francis, 1996, pp. 189–194.

[25] A. Bangor, P. T. Kortum, and J. T. Miller, “Determining what individual SUS scores mean: Adding an adjective rating scale,” *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.

[26] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

A. Participant Information Sheet and Consent

Participant Information Summary

Participant-facing information for the Cycle I written questionnaire and semi-structured interviews is summarised below.

Topic	Information Provided
Study purpose	The study examines productivity limitations experienced by embedded software engineers when developing C/C++ code within PLUS+1 GUIDE and evaluates whether a VS Code extension could meaningfully improve the workflow.
Participant contribution	Participants were told that their experience and honest opinions mattered, and that detailed answers would directly shape the design of the PLUS+1 Developer Tools extension.
Written questionnaire format	The questionnaire contained 15 questions across seven sections. Participants were asked to answer openly and specifically, with no right or wrong answers, and could skip questions that did not apply to their role.
Interview format	The semi-structured interview covered the same problem area in conversational form. Participant answers were used to inform the design of the extension.
Voluntary participation	Participation was entirely voluntary. Participants could skip questions, stop participation, or withdraw at any time without consequence.
Anonymisation	Names would not appear in the thesis or any publication. Participants would be referred to only by participant identifiers such as P1 or P2.
Audio recording	Interview sessions would be audio-recorded only if explicit consent was provided. If not, the interviewer would take written notes instead.
Data storage	Questionnaire responses, recordings, and transcripts would be stored in password-protected folders accessible only to the thesis authors and academic supervisor, and deleted after thesis submission.
Confidentiality	Danfoss-specific technical details would be reported only at an abstraction level that protects company confidentiality.
Contact	Questions about the study could be sent to Razan Kngo (`rk222rr@student.lnu.se`) or Marios Gkikopouli (`mg223uk@student.lnu.se`).

Consent Declarations Included in the Information Sheet

Instrument	Consent Wording / Confirmation Items
Written questionnaire	By submitting the form, participants confirmed that they had read and understood the information, understood participation was voluntary and withdrawal was possible, and consented to anonymised responses being used in the thesis and related publications.
Semi-structured interview	Before the interview began, participants were asked to confirm that they had read and understood the information, understood participation was voluntary and withdrawal was possible, consented or did not consent to audio recording, and agreed to take part in the interview.
Interview signature fields	The interview consent form included fields for participant signature, date, and participant ID for researcher use.

B. Consent Forms

Consent Form Summary

Consent confirmations used across the empirical activities are summarised below.

Study Activity	Consent Confirmation
Cycle I written questionnaire	By submitting the form, participants confirmed that they had read and understood the participant information, understood that participation was voluntary and withdrawal was possible, and consented to anonymised responses being used in the thesis and related publications.
Cycle I semi-structured interview	Participants confirmed before the interview that they had read and understood the information, understood that participation was voluntary and withdrawal was possible, indicated whether they consented to audio recording, and agreed to take part in the interview. The form included fields for participant signature, date, and participant ID for researcher use.
Cycle III prototype evaluation	Participants confirmed that they had previously participated in the first survey and had been informed of the general research purposes, understood that responses would be used for academic research as part of a master's thesis at Linnaeus University, understood that responses were anonymous within the research team and would not be shared with their employer or manager, confirmed that they had attempted the Part A tasks using the extension before completing the survey questions, and understood that they could withdraw from the survey at any point without consequence.

C. Interview Guide

Semi-Structured Interview Guide

Instrument: Semi-structured interview guide for Cycle I problem investigation.
Project: PLUS+1 Developer Tools for VS Code.
Institutional context: Linnaeus University and Danfoss Power Solutions AB.
Purpose: To understand productivity limitations experienced by embedded software engineers when developing C/C++ code within PLUS+1 GUIDE, and to explore whether a VS Code extension could meaningfully improve the workflow. Participant answers were used to shape the design of the PLUS+1 Developer Tools extension.

The guide was used flexibly. Interviewers were instructed not to read probing questions unless a participant's answer was too brief or vague, to let participants speak freely, and to allow 5–8 seconds of silence before prompting.

Participant Rights, Consent, and Interview Protocol

Topic	Information / Instruction
Voluntary participation	Participation was entirely voluntary. Participants could stop at any time or skip any question without consequence.
Anonymisation	Names do not appear in the thesis or publications. Participants are referred to by identifiers such as P1 and P2.
Audio recording	Sessions were audio-recorded only with explicit consent. If consent was not given, written notes were taken instead.
Data storage	Recordings and transcripts were stored in password-protected folders accessible only to the thesis authors, and deleted after submission.
Confidentiality	Danfoss-specific technical details were reported only at a level of abstraction that protects company confidentiality.
RQ mapping	`[RQ1]` denotes problem investigation/current limitations; `[RQ2]` denotes solution-design input; `[RQ3]` denotes evaluation, adoption, and artefact requirements.

Interview Structure Overview

Part	Duration	Purpose
Theme A – Background	~5 min	Establish rapport, confirm role and experience, and explain the interview structure.
Theme B – Daily Workflow	~8 min	Capture an end-to-end development session and expose tool-switching behaviour.
Theme C – Pain Points & Limitations	~15 min	Gather specific, empirically grounded accounts of where GUIDE falls short for C/C++ work.
Theme D – Build Process	~5 min	Understand build initiation and identify build-related friction points.
Theme E – Modern Tooling	~7 min	Identify valued modern-editor features and surface safety or workflow constraints.
Theme F – Openness to Change	~7 min	Assess adoption willingness and establish the minimum viable feature set.
Open Feedback & Closing	~5 min	Allow participants to raise issues not covered by earlier questions.

Interview Questions

Table C.0.1: Cycle I Semi-Structured Interview Questions

Theme	Q	RQ	Main Question	Probing Questions / Interviewer Prompts
A – Background	1	RQ1	How long have you been working with PLUS+1 GUIDE?	How has your use of GUIDE changed over that time? Do you consider yourself an experienced GUIDE user?
A – Background	2	RQ1	How would you describe your role in relation to C/C++ development within GUIDE projects?	Examples included primarily C/C++ development, mixed development, or occasional use. Prompts covered the approximate percentage of GUIDE work involving C/C++ versus graphical/dataflow components, and whether the participant works alone or in a team sharing project files.
A – Background	3	RQ1, RQ2	Have you used modern code editors such as Visual Studio Code, CLion, or similar tools for other projects outside of GUIDE?	Which editors have you used and for what kinds of projects? What features do you use most regularly? How does the experience compare to working in GUIDE?
B – Daily Workflow	4	RQ1	Describe what a typical session of writing or editing C/C++ code looks like within GUIDE, from opening a project to completing a code change.	How do you open and navigate to the file? How do you know when the change is correct; do you compile frequently? How long does a typical change-to-build cycle take? Is there anything in that sequence that should not be necessary?
B – Daily Workflow	5	RQ1	During a typical development session, do you switch between GUIDE and other tools such as a separate editor, file explorer, terminal, or browser? If yes, which tools and why?	How many times per session do you estimate you switch tools? Is context-switching disruptive? Have you tried to reduce this switching, and if so how?

(continued on next page)

(continued from previous page)

Theme	Q	RQ	Main Question	Probing Questions / Interviewer Prompts
C – Pain Points	6	RQ1	What aspects of writing or editing C/C++ code in GUIDE are most time-consuming or frustrating?	Can you give a recent specific example? How much time per day or week would you estimate this costs? Is this something you have raised internally before?
C – Pain Points	7	RQ1	How do you navigate code in GUIDE, for example jumping to a function definition, finding all usages of a variable, or searching across files? Are you satisfied with how this works?	What do you do when you need to find where a function is defined? How do you search for a string or symbol across all project files? Have you spent significant time looking for something a modern editor would have found instantly?
C – Pain Points	8	RQ1	How does GUIDE provide feedback on code errors or issues while you are writing code? Do you discover errors late, for example only at build time?	Does GUIDE highlight errors as you type, or only when compiling? Can you give an example of an error caught very late that real-time diagnostics would have caught immediately? How does late error discovery affect your development rhythm?
C – Pain Points	9	RQ1	How do you perform refactoring tasks, such as renaming a variable or restructuring code? Are there limitations you regularly encounter?	How do you currently rename a variable used in many places? Do you use find-and-replace, and what are the risks? How often do refactoring limitations cause you to avoid changes you would otherwise make?
D – Build Process	10	RQ1, RQ2	How do you currently initiate builds for PLUS+1 projects? Are there inefficiencies or pain points in this process?	Do you always build from within GUIDE, or ever use command-line tools? How long does a typical build take? Is incremental build supported? Are build errors reported clearly? Would it be useful to trigger builds from VS Code without switching to GUIDE?
E – Modern Tooling	11	RQ1, RQ2	Which modern-editor features are most valuable but missing from the GUIDE workflow, or would benefit your work if you have not used modern editors?	How important is IntelliSense/code completion? How valuable would real-time error highlighting without compiling be? Would integrated Git tooling improve the workflow? Are AI-assisted features such as GitHub Copilot desirable? What are the top three features?
E – Modern Tooling	12	RQ2, RQ3	Are there modern-editor features that would be inappropriate or risky in a safety-critical embedded development context?	Would AI-generated code be concerning in safety-critical components? Are there features that could introduce hidden changes to the `.plx` project structure? What level of tool validation or certification would be expected before adoption?
F – Openness to Change	13	RQ3	How would you feel about editing C/C++ code in VS Code while continuing to use GUIDE for graphical design, simulation, and final builds?	What would need to be true about synchronisation between VS Code and GUIDE for you to trust it? Would you be concerned about the tools getting out of sync? How would your team react? Would management or safety-compliance requirements affect adoption?

(continued on next page)

(continued from previous page)

Theme	Q	RQ	Main Question	Probing Questions / Interviewer Prompts
F – Openness to Change	14	RQ2, RQ3	What would a VS Code plugin for PLUS+1 need to include at minimum to genuinely improve your daily workflow and be worth adopting?	Is there a single feature that would make you start using it immediately? Are there features without which the plugin would not be useful? How important is reliability compared with having many features? Would you participate in testing early versions?
Open Feedback	15	RQ1, RQ2, RQ3	Is there anything else about your experience developing with PLUS+1 GUIDE that is important for us to understand?	Is there a problem you expected us to ask about that we did not? Is there anything you specifically want to make sure ends up in the thesis?

Post-Session Interviewer Notes

Field	Purpose
Overall impression	Record the overall impression of the session.
Key themes / pain points	Note the themes or pain points that emerged most strongly.
Unexpected points	Capture anything the participant said that surprised the interviewer.
Follow-up questions	List questions to explore in later sessions.
Non-verbal observations	Note hesitation, frustration, enthusiasm, or other relevant non-verbal observations.
Filing and coding	Complete notes within 2 hours of session end. Save as `INT-[ParticipantID]-[YYYYMMDD]-notes.pdf`. Begin thematic coding within 48 hours using the shared coding spreadsheet columns: Participant ID, Quote, Initial Code, Theme, and RQ.

D. Self-Completion Questionnaire

PLUS+1 Developer Experience Survey

Instrument: Written self-completion questionnaire for Cycle I problem investigation.
Project: PLUS+1 Developer Tools for VS Code.
Institutional context: Linnaeus University and Danfoss Power Solutions AB, 2026.
Purpose: To understand productivity limitations experienced by embedded software engineers when developing C/C++ code within PLUS+1 GUIDE, and to evaluate whether a VS Code extension could meaningfully improve the development workflow.

Participants were asked to answer openly and specifically. They were informed that there were no right or wrong answers, that detailed experience would directly shape the design of the PLUS+1 Developer Tools extension, and that any question could be skipped if it did not apply to their role. The questionnaire contained 15 questions across seven thematic sections.

Consent, Data Use, and Participant Details

Topic	Information Provided to Participants
Voluntary participation	Completing the questionnaire was entirely voluntary. Participants could skip any question or stop at any time.
Anonymisation	Names would not appear in the thesis or any publication. Participants would be referred to only by participant codes such as P1.
Data storage	Responses would be stored in a password-protected folder accessible only to the thesis authors and supervisor, and deleted after thesis submission.
Confidentiality	Danfoss-specific technical details would be reported only at an abstraction level that protects company confidentiality.
Consent statement	Submission confirmed that the participant had read and understood the information, understood participation was voluntary and withdrawal was possible, and consented to anonymised use of responses in the thesis and related publications.
Contact	Razan Kngo (`rk222rr@student.lnu.se`) and Marios Gkikopouli (`mg223uk@student.lnu.se`).
Participant details collected	Role/job title; department/team; years using PLUS+1 GUIDE. These details were used for context only and were anonymised in written outputs.

Questionnaire Structure

Table D.0.1: Cycle I Self-Completion Questionnaire Items

Section	Q	Main Question	Optional Prompts / Context Given to Participants
A – Background	1	How long have you been working with PLUS+1 GUIDE?	How has your use of GUIDE changed over that period? How would you describe your overall level of experience with GUIDE?

(continued on next page)

(continued from previous page)

Section	Q	Main Question	Optional Prompts / Context Given to Participants
A – Background	2	How would you describe your role in relation to C/C++ development within GUIDE projects?	For example, primarily C/C++ development, mixed development, or occasional use. Roughly what percentage of GUIDE work involves writing or editing C/C++ code versus graphical/dataflow components? Do you typically work alone on components, or share project files with other developers?
A – Background	3	Have you used modern code editors such as Visual Studio Code, CLion, or similar tools for projects outside of GUIDE? If so, which ones?	What features do you use most regularly in those editors? How does the editing experience compare to working in GUIDE?
B – Daily Workflow	4	Describe what a typical session of writing or editing C/C++ code looks like for you within the GUIDE environment. Walk through the steps you usually take from opening a project to completing a code change.	How do you open and navigate to the file you need to edit? How often do you compile during a session? How long does a typical change-to-build cycle take? Is there any step in that sequence that you feel should not be necessary?
B – Daily Workflow	5	During a typical development session, do you switch between GUIDE and other tools, such as a separate text editor, file explorer, terminal, or browser? If yes, please describe which tools and why you switch to them.	Roughly how many times per session do you switch tools? Does switching disrupt concentration or workflow? Have you tried to reduce this switching in any way?
C – Pain Points	6	What aspects of writing or editing C/C++ code in GUIDE do you find most time-consuming or frustrating? Please be as specific as possible.	This was presented as part of the most important section. Participants were asked for concrete examples and estimated time costs. Prompts included: Can you describe a recent specific example? How much time per day or week would you estimate this costs you?
C – Pain Points	7	How do you navigate code in GUIDE, for example jumping to a function definition, finding all usages of a variable, or searching across files? Are you satisfied with how this works?	What do you do when you need to find where a function is defined? How do you search for a string or symbol across all files in a project? Have you ever spent significant time finding something that a modern editor would have located instantly?
C – Pain Points	8	How does GUIDE provide feedback on code errors or issues while you are writing code? Do you find yourself discovering errors late in the process, for example only at build time?	Does GUIDE highlight errors as you type, or only when you compile? Can you describe an example of an error caught very late that an IDE with real-time diagnostics would have caught immediately? How does late error discovery affect your development rhythm?
C – Pain Points	9	How do you perform refactoring tasks, such as renaming a variable or restructuring code? Are there limitations you regularly encounter?	How do you currently rename a variable that is used in many places? What are the risks of your current approach? Do refactoring limitations ever cause you to avoid making changes you would otherwise make?

(continued on next page)

(continued from previous page)

Section	Q	Main Question	Optional Prompts / Context Given to Participants
D – Build Process	10	How do you currently initiate builds for your PLUS+1 projects? Are there inefficiencies or pain points in this process you would like to highlight?	Do you always build from within GUIDE, or sometimes use command-line tools? How long does a typical build take? Is incremental build supported? How are build errors reported, and are they clear and easy to act on? Would it be useful to trigger builds directly from a code editor without switching to GUIDE?
E – Modern Tooling	11	If you have used modern editors such as VS Code or JetBrains tools, which features do you find most valuable that are currently missing from your GUIDE workflow? If you have not used modern editors, which features do you feel would benefit your work most?	How important is IntelliSense/code completion? How valuable would real-time error highlighting without compiling be? Would integrated Git tooling improve the workflow? If you had to pick your top three features, what would they be?
E – Modern Tooling	12	Are there any modern editor features that you feel would be inappropriate or risky to use in a safety-critical embedded development context? Please explain your reasoning.	Would you be concerned about AI-assisted code generation in safety-critical components? Are there features that could introduce hidden or unintended changes to the `.plx` project structure?
F – Openness to Change	13	How would you feel about editing C/C++ code in VS Code while continuing to use GUIDE for graphical design, simulation, and final builds? What concerns, if any, would you have?	What would need to be true about synchronisation between VS Code and GUIDE for you to trust it? Would you be concerned about the two tools getting out of sync? How do you think your team would react to such a change?
F – Openness to Change	14	What would a VS Code plugin for PLUS+1 need to include at minimum to genuinely improve your daily workflow and be worth adopting?	Is there a single feature that, if present, would make you start using it immediately? Are there features without which the plugin would not be useful at all? How important is reliability versus having a wide range of features?
G – Open Feedback	15	Is there anything else about your experience developing with PLUS+1 GUIDE that you think is important for us to understand?	Is there a problem you expected to be asked about that was not covered? Is there anything you would specifically want to make sure is reflected in this research?

E. Cycle I Requirements and Pain-Point Evidence

E.1 User-Story Requirements (REQ-01–REQ-49)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-01	Code Intelligence	As a developer, I want C/C++ syntax and basic semantic errors highlighted in real time as I type, so that I catch mistakes before committing to a multi-minute compile.	Missing semicolons, unclosed brackets, and undeclared identifiers flagged inline while typing; findings appear in the VS Code Problems panel; no compile step required; works on all C/H files in a GUIDE project.	Must	No	P1–P4, P6–P10
REQ-02	Code Intelligence	As a developer, I want intelligent auto-completion of functions, variables, and struct members, so that I write code faster with fewer typos.	Function-name completion with signature preview; variable and parameter completion in scope; struct-member completion via `.` and `->`; completion draws from project headers and linked standard library.	Must	No	P1–P4, P6, P7, P9, P10
REQ-03	Code Intelligence	As a developer, I want hover tooltips showing type, signature, and documentation for any symbol, so that I don't have to leave my current position to look things up.	Hover shows function signature and parameter names; hover shows type of variables and constants (including `#define` values); hover shows doc comments where present; works for both project symbols and standard library.	Must	No	P2–P4, P6, P7, P9
REQ-04	Code Intelligence	As a developer, I want Ctrl+click / F12 to jump to a symbol's definition or declaration, so that I can navigate large codebases without manual searching.	Ctrl+click navigates to definition across project files; separate command for go-to-declaration vs. go-to-definition; works for functions, variables, types, and macros.	Must	No	P2–P4, P6, P7, P9, P10
REQ-05	Code Intelligence	As a developer, I want to find all references to a symbol across the project, so that I can assess impact before changing or removing it.	Right-click → Find All References enumerates every usage; results are clickable and jump to source location; covers all C/H files in the GUIDE project.	Must	No	P2–P4, P6, P7, P9, P10

(continued on next page)

(continued from previous page)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-06	Code Intelligence	As a developer, I want selecting an identifier to automatically highlight its other occurrences in the file, so that I can visually trace a variable's use at a glance.	Selecting a symbol highlights all matching occurrences in the active file; highlighting is distinct from search highlighting; toggleable in settings.	Should	No	P1, P7
REQ-07	Search & Replace	As a developer, I want fast find-in-files across the project, so that locating code doesn't cost me minutes per search like GUIDE's current Find tool.	Ctrl+Shift+F searches all project C/H files; results returned in under a second for typical projects; results clickable with line preview; regex and case-sensitivity options.	Must	No	P3, P4, P6, P9, P10
REQ-08	Search & Replace	As a developer, I want search-and-replace with preview and undo, so that I can bulk-rename safely without fear of unintended changes.	Preview all matches before replacing; replace per-occurrence, per-file, or all; regex and whole-word options; undo reverses the full operation.	Must	No	P1, P3, P6
REQ-09	Refactoring	As a developer, I want a symbol-rename refactor that safely updates every reference to a variable or function across files, so that renaming stops being a manual, error-prone chore.	Rename Symbol updates all references in C/H files in one action; preview before applying; scope-aware (local vs. global); surfaces manual follow-ups where a rename crosses into GUIDE-owned artefacts (e.g. POU interface wire).	Should	Partial	P3, P6, P8–P10
REQ-10	Refactoring	As a developer, I want keyboard shortcuts to toggle block comment / uncomment, so that I can experiment and roll back quickly.	Shortcut comments/uncomments the current selection; works on multi-line selections; handles both `//` and `/* */` styles.	Could	No	P6
REQ-11	Refactoring	As a developer, I want to fold and unfold functions, comments, and regions, so that I can focus on the code I'm editing.	Fold markers in the gutter for functions, blocks, and multi-line comments; fold-all / unfold-all commands; folding state optionally persisted per file.	Could	No	P6

(continued on next page)

(continued from previous page)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-12	Readability	As a developer, I want clear syntax highlighting with bracket matching and colorization, so that I can read and edit nested code without losing my place.	Standard C/C++ syntax coloring; matching bracket highlight when cursor is adjacent; rainbow / paired bracket coloring available; consistent with user-selected VS Code theme.	Must	No	P1, P2, P4, P6, P9
REQ-13	Readability	As a developer, I want to switch between dark and light themes, so that I can use dark while coding and light when capturing code for documentation or slides.	Inherits VS Code's theme settings natively; plugin-specific UI respects active theme.	Could	No	P6
REQ-14	GUIDE Integration	As a developer, I want file changes saved in VS Code to be picked up by GUIDE without per-file Reload prompts, so that the iteration loop isn't interrupted by pop-ups.	GUIDE detects VS Code saves automatically; no per-file reload dialog; no close/reopen GUIDE workflow required for file swaps; concurrent edits in GUIDE and VS Code are reconciled safely (last-writer-wins with warning).	Must	Partial	P1–P3, P7, P8, P10
REQ-15	GUIDE Integration	As a developer, I want to trigger a GUIDE build from VS Code, so that I don't have to alt-tab to GUIDE for every compile cycle.	Command initiates a GUIDE build; build output streamed into VS Code; build can target current project without requiring GUIDE window focus.	Should	No	P1, P2, P7–P10
REQ-16	GUIDE Integration	As a developer, I want to compile C code (or only changed modules) without triggering a full GUIDE build, so that I validate C changes in seconds instead of minutes.	C-only compile command available; does not regenerate unchanged SCS modules or SDLs; errors reported in VS Code within seconds for typical library-size changes.	Should	No	P2, P4, P9
REQ-17	GUIDE Integration	As a developer, I want GUIDE compiler errors to link directly to the originating C source in VS Code (not to generated files), so that I can jump to the real problem.	Clicking an error opens the original C/H file at the correct line in VS Code; POU compile errors resolve to the POU XML source, not the generated `plcmain.c`; generated-file indirection is hidden from the developer.	Must	Partial	P2–P4, P8, P9

(continued on next page)

(continued from previous page)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-18	GUIDE Integration	As a developer, I want C code errors to appear in the standard Problems panel, not only in the build log, so that they aren't missed.	Errors and warnings populate the Problems panel with severity, file, and line; errors from IDL / POU interface code also reach the Problems panel; double-click navigates to source.	Should	Partial	P1, P2
REQ-19	POU & Interface	As a developer, I want to edit POU code in VS Code with proper syntax support, so that I don't have to fall back to GUIDE's editor for POU work.	POU (XML) files editable in VS Code; syntax-aware rather than raw XML; changes round-trip cleanly to GUIDE.	Should	Partial	P2, P5, P6, P8
REQ-20	POU & Interface	As a developer, I want to define and edit POU interfaces (inputs/outputs/types) from VS Code, so that the full C-integration workflow lives in one tool.	POU interface editable in VS Code; type selection, direction (in/out), and naming all supported; changes synchronized to GUIDE project.	Should	Partial	P3, P5
REQ-21	POU & Interface	As a developer integrating a new C file into a GUIDE project, I want to auto-generate a POU wrapper for an existing C function, so that I don't spend time on monotonous boilerplate.	Select a C function and generate a POU wrapper in one action; wrapper interface derived from function signature; manual review before commit.	Could	Partial	P3, P5, P9, P10
REQ-22	POU & Interface	As a developer, I want linting and static analysis to run against POU code inside VS Code, so that I catch issues without tricks like extracting POU text to a temp file.	Static analysis runs on POU code in-place; findings appear in Problems panel; no scripting workaround needed.	Should	Yes	P2, P8
REQ-23	POU & Interface	As a developer, I want readable diffs of POU code (not raw XML), so that code reviews don't require tooling workarounds.	Diff view renders POU code as code, not XML; line-level diff granularity.	Could	Partial	P8

(continued on next page)

(continued from previous page)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-24	Version Control	As a developer using Git, I want integrated Git operations (pull, push, commit, branch, diff, compare), so that I don't need a separate Git tool.	Full set of Git commands reachable from VS Code; diffs work cleanly on C/H files; Git history browsable per file.	Should	No	P2, P4, P5, P7
REQ-25	Version Control	As a developer on a team using SVN, I want SVN operations available in the plugin, so that my workflow isn't second-class because my team hasn't moved to Git.	Basic SVN operations (commit, update, diff, revert, log); file history browsing; integration with TortoiseSVN acceptable.	Should	No	P1, P6–P8
REQ-26	Version Control	As a developer, I want to rename files in VS Code without breaking Git/SVN history or corrupting GUIDE's project tracking, so that refactoring doesn't cost me my history.	File rename uses the VCS rename operation (preserving history); GUIDE project file updated consistently with the rename; no orphaned references in the project tree.	Could	Partial	P7
REQ-27	Debugging	As a developer, I want to set and hit breakpoints from VS Code while running GUIDE-built code, so that I don't have to switch tools for runtime debugging.	Set/toggle breakpoints in VS Code; variable inspection in VS Code's debug view; step in / over / out; hooks into GUIDE's existing debug target.	Could	Partial	P5, P7–P10
REQ-28	Static Analysis	As a developer, I want integrated static analysis (linting) for C code with awareness of GUIDE-specific configuration, so that I catch coding-standard and safety issues early.	Lint runs on save or in real time; consumes GUIDE-specific lint configuration files; findings populate the Problems panel with severity and rule ID.	Must	Yes	P2, P4, P8
REQ-29	Static Analysis	As a developer, I want consistent code formatting applied automatically or on demand, so that reviews focus on logic rather than style.	Format-on-save (opt-in); format-document command; follows team-configurable style.	Could	No	P1, P4

(continued on next page)

(continued from previous page)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-30	AI Assistance	As a developer, I want AI-assisted features (e.g. Copilot) that I can cleanly toggle on and off, so that I control when AI is active for energy, IP, or safety reasons.	Toggle AI on/off; control is per-session or persistent in settings.	Could	Yes	P1, P4–P6
REQ-31	AI Assistance	As a senior developer, I want AI assistance that understands GUIDE-specific formats (IDL, POU XML, SCS), so that Copilot-style help extends to interface and test-case work, not just plain C.	AI context includes IDL, POU XML, and relevant SCS-derived artefacts; AI can draft unit tests referencing GUIDE interfaces; generated output clearly labeled as AI-produced.	Could	Yes	P4
REQ-32	Safety & Reliability	As a safety-responsible engineer, I want AI-generated code in safety-critical contexts clearly flagged and gated on explicit review, so that we maintain our safety posture.	AI-generated code flagged and requires explicit review before integration into safety-critical contexts.	Should	Yes	P2, P5–P7, P9, P10
REQ-33	Safety & Reliability	As a developer, I want the plugin's core features to be reliable even if advanced features are less so, so that I can trust the tool for daily work.	Core features (editing, save, sync with GUIDE, find, go-to-definition) meet a higher reliability bar; advanced features may carry a lower bar but must fail gracefully; no data loss on crash and unsaved state recovery.	Must	Yes	P1, P6, P7
REQ-34	Safety & Reliability	As a developer working on certified or released projects, I want the plugin to support older, certified GUIDE versions, so that I don't have to upgrade the toolchain to use the editor.	Plugin works against previously-certified GUIDE versions (list to be agreed with product); no silent behavioural changes when project targets an older GUIDE version; feature-gating where a feature is not available on an older version.	Must	Yes	P7

(continued on next page)

(continued from previous page)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-35	Adoption	As a team lead rolling the plugin out, I want demo-able workflows and a named support contact, so that team members adopt it without a steep learning curve.	Getting-started walk-through / demo project included; clear escalation path when things break; documentation assumes a GUIDE-expert, occasional-C-user reader.	Should	No	P1
REQ-36	GUIDE Integration	As a developer, I want a project tree in VS Code that mirrors GUIDE's Project Manager, so that I can navigate the PLUS+1 project structure from the editor without switching back to GUIDE.	Tree view reflects the GUIDE project hierarchy (C/H files, POUs, libraries); tree reads from the `.p1x` project file; click-to-open for C/H files; tree refreshes when the project file changes on disk.	Must	No	P9, P10
REQ-37	GUIDE Integration	As a developer, I want to add, remove, and rename C/H files directly from VS Code with the `.p1x` project file updated automatically, so that purely C-focused tasks don't require opening GUIDE at all.	Add / remove / rename actions available from the VS Code project tree; `.p1x` updated atomically on each change (no partial writes); operation works without GUIDE running; GUIDE reconciles the `.p1x` state on next load without manual intervention.	Must	Partial	P9, P10
REQ-38	GUIDE Integration	As a developer alternating between VS Code and GUIDE, I want each tool to notice when the other has modified shared artefacts, so that concurrent edits are surfaced instead of silently lost.	VS Code detects GUIDE-side changes to open files or project structure and prompts the user; conflicting concurrent edits produce a visible warning, not silent loss; sync status indicator visible in VS Code.	Must	Partial	P10
REQ-39	POU & Interface	As a developer integrating a new C file into a GUIDE project, I want a guided workflow covering C-file addition, C Code POU wrapper creation, and the 'Make available to PLC C code POUs' step, so that I don't have to discover the multi-step ritual by trial and error.	Command-palette flow walks through each required step in order; each step is labeled and explains which GUIDE artefact it produces; defaults derived from the selected C function signature where possible; manual fall-back available for advanced users; flow completes end-to-end without opening GUIDE.	Should	Partial	P9, P10

(continued on next page)

(continued from previous page)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-40	Readability	As a developer, I want to easily zoom in/out and adjust text size in the editor, so that the UI stays comfortable as eyesight or display setup changes.	Keyboard shortcut zooms text up/down; persistable font-size setting per workspace; plugin's own UI surfaces also respect the zoom/font settings.	Could	No	P9
REQ-41	GUIDE Integration	As a developer iterating rapidly on C code changes, I want GUIDE project builds to complete quickly (or at least incrementally, recompiling only what changed), so that minutes-long compile waits don't interrupt my flow 5–10 times an hour.	Incremental builds skip unchanged SCS modules and auto-generated code; build progress is visible (stage, elapsed time); typical full-build time target aligned with project size, not hard-coded minutes.	Must	No	P2, P4, P6, P8, P9
REQ-42	Version Control	As a developer on a team, I want to merge concurrent changes to graphical (SCS) code without splitting work into separate blocks and reconciling manually, so that collaboration on GUIDE projects doesn't need off-tool workarounds.	Side-by-side diff for SCS files; three-way merge with conflict markers; operates at graphical-semantic level, not raw file bytes.	Should	Partial	P1, P4
REQ-43	POU & Interface	As a developer, I want to call C functions directly from GUIDE code without wrapping each one in a C Code POU, so that the monotonous wrapper step stops slowing down C integration.	C functions callable from GUIDE without manual POU creation; existing POU-based code continues to work (backwards compatible); type-checking enforced at the GUIDE/C boundary.	Should	Partial	P3, P5, P10
REQ-44	GUIDE Integration	As a developer integrating a third-party library into a GUIDE project, I want folder structures preserved rather than flattened, so that I can adopt external libraries without first restructuring them.	Imported library folders preserved inside the GUIDE project; build paths resolve correctly across subfolders; VS Code project tree reflects the preserved structure.	Could	No	P4

(continued on next page)

(continued from previous page)

ID	Category	User Story	Acceptance Criteria	MoSCoW	Safety	Source
REQ-45	Code Intelligence	As a developer using standard C, I want `<stdbool.h>` and GUIDE's NV `bool` struct not to collide by name, so that I can use standard types without workarounds or accidental redefinitions.	No name collision when including `<stdbool.h>`; existing NV `bool` usage continues to compile; clear diagnostic when both are used conflictingly.	Could	Partial	P7
REQ-46	GUIDE Integration	As a developer working on CAN-bus applications, I want to manage DBC files inside the PLUS+1 toolchain rather than switching to Vector DBC++, so that CAN configuration stays in one environment.	Open, edit, and save DBC files; integration with GUIDE's CAN message definitions; DBC validation feedback in the editor.	Could	No	P6
REQ-47	GUIDE Integration	As a developer, I want GUIDE code pages and Service Tool diagnostic pages to share a synchronized namespace, so that diagnostic signals don't have to be maintained manually in two places.	Namespace changes propagate automatically between the two tools; conflicts surfaced as warnings, not silent overwrites; audit log of propagation events.	Could	No	P6
REQ-48	Readability	As a developer, I want to auto-generate code documentation from my C code (Doxygen-style), so that reference docs stay in sync with the code without manual effort.	Doc-comment templates inserted via command/keyboard shortcut; generate HTML or Markdown documentation from annotated C code; documentation preview available in VS Code.	Could	No	P6
REQ-49	GUIDE Integration	As a developer investigating a graphical signal, I want one-click navigation to the source that powers or sets a wire's value, so that I don't spend time scanning 20+ pages to find it.	Right-click / context action on a wire → jump to its source component; works across module boundaries without manual reload; navigation history for back/forward traversal.	Should	No	P3, P4

E.2 Disposition of Requirements with Respect to the Cycle II Prototype

Dispositions are reported as **Delivered**, **Partial**, **Future Work**, **Out of Scope**, or **Uncertain**.

ID	Disposition	Notes and Representative Quotes	Internal Notes
REQ-01	Delivered (I2)	Most-cited pain point across the survey; GUIDE currently surfaces many errors only at build time, and builds can take 3–20 minutes.	
REQ-02	Delivered (I2)	P1 explicitly lists auto-complete in top requested features; P3 calls IntelliSense a base requirement.	
REQ-03	Delivered (I2)	P7 ranks hover documentation in top 3.	
REQ-04	Delivered (I2)	P7 ranks Ctrl+click to source as #1 missing feature.	
REQ-05	Delivered (I2)	(Cited by 8 of 10 respondents alongside REQ-04.)	
REQ-06	Delivered (I2)	P1 specifically calls this out as a valued Notepad++ feature.	
REQ-07	Delivered (I2)	P4: ‘Searching with the Find Tool in our very large project is very slow. So I usually try to avoid using it.’ P6 calls the existing search ‘extremely time consuming’.	
REQ-08	Delivered (I2)	P1: ‘I use the replace and go through it step by step to make sure I don’t do unwanted changes.’	
REQ-09	Future Work	P8: ‘If I update a lot of signals then I might just use find and replace inside vscode or notepad++ directly in the XML.’ Safety flagged Partial because rename errors in signal names can have runtime consequences.	
REQ-10	Delivered (I2)	(Standard VS Code keyboard shortcut; no custom implementation required.)	
REQ-11	Delivered (I2)	(Standard VS Code folding; no custom implementation required.)	
REQ-12	Delivered (I2)	P1 specifically names ‘coloring brackets’ as one of the most important editor features.	
REQ-13	Delivered (I2)	(Inherits VS Code theme natively.)	
REQ-14	Future Work	P1 describes the current swap workflow (remove, save, close GUIDE, reopen, copy, add, save) as a clear improvement target. Safety Partial because file-sync issues can silently lose changes.	GUIDE will ask for reload only if a project is modified in VS Code while having GUIDE open at the same time with that project.
REQ-15	Delivered (I3)	P8: ‘It would be nice to trigger a build without having to switch back to my active GUIDE instance.’	

(continued on next page)

(continued from previous page)

ID	Disposition	Notes and Representative Quotes	Internal Notes
REQ-16	Future Work	P2 explicitly asks for C-only compile because full builds take 15–20 minutes on some projects. P4: 'I wish GUIDE was a bit smarter when it compiles so that it does not need to generate code for all the modules.'	Maybe (depends on GUIDE CLI support).
REQ-17	Partial (I3)	P3: '...it doesn't properly validate the POU before it packages every POU to a `plcmain.c` then it finds the error in `plcmain.c` and reports that out as opposed to telling you what PLC, what POU, and what line.' Safety Partial because unclear error origin delays fixes in certified work.	Implemented for C compile errors. Has not been tested for POU errors, etc.
REQ-18	Partial (I3)	P1: 'some errors are not shown in the error tab but in the build tab. This specifically happens when doing something wrong in IDL files.'	Implemented for C compile errors. Has not been tested for POU errors, etc.
REQ-19	Future Work	P2: 'Also being able to edit POUs outside of GUIDE would be nice.' P5 explicitly doesn't want to touch the POU editor.	
REQ-20	Future Work	P5: 'I don't really need much beyond what VSCode provides. Just don't make me use the POU interface.' P3: 'having the ability to setup POU interfaces in VSCode would be great.'	
REQ-21	Future Work	P3 explicitly notes this is already often done with AI prompts; wants it first-class.	
REQ-22	Future Work	P2: 'Currently trying to Lint POUs isn't very easy without adding code to them.' P8 maintains a script to extract POU code for diffing.	
REQ-23	Future Work	P8 uses a custom extraction script; review diff quality affects safety code review.	
REQ-24	Delivered (I2)	P5: 'Unless the Git tooling was fully featured (pull, fetch, push, commit, add, compare, switch, etc.) I'd just keep using my other one.' P4 already uses Git with GUIDE projects.	
REQ-25	Uncertain	Tension with REQ-24 noted: P1 explicitly says 'many projects are in SVN due to using GUIDE and low acceptance of GIT from many coworkers.' Both must be supported.	Might work by using existing plug-ins in VS Code.
REQ-26	Uncertain	P7: 'Renaming files can be a little bit of headache to keep git/svn history and not have GUIDE get mad.'	Should work in theory but has not been tested.

(continued on next page)

(continued from previous page)

ID	Disposition	Notes and Representative Quotes	Internal Notes
REQ-27	Delivered (I4)	P7: 'Being able to set debugger points in vscode would be nice and have that connection with the tool.' Lower priority than code-intelligence basics; higher complexity.	If it means being able to run debug in VS Code.
REQ-28	Future Work	P8 runs LINT manually before compiling; core to safety-critical workflows (MISRA-class rules).	Should be checked. Might be possible with existing VS Code linting tools and manual config.
REQ-29	Future Work	P1 lists 'format code easily' as a valued editor feature.	Should be checked. Might be possible with existing VS Code tools.
REQ-30	Out of Scope	P1: 'AI that can be turned on or off easily would be great' (energy concern). P6: AI 'up to now is not reliable enough to be used in our context, especially if safety-critical'.	VS Code already has integration.
REQ-31	Out of Scope	P4 is already doing this with ad-hoc prompts and Ratatosk-generated C/H files.	
REQ-32	Out of Scope	P5: 'I would not use AI for broad development of safety code. But I may use it to write very targeted functions with strong coding guideline requirements and heavy review by myself.' P7: would not trust AI 'to create more than 10 lines of code with safety critical or cybersecurity related code.'	
REQ-33	Delivered (I1–I4)	P1: 'Reliability is more important than a wide range of features.' P7 explicitly allows a tiered reliability bar.	
REQ-34	Future Work	P7: 'I often work in the previous year's certified version for development, so backwards compatibility would be a must.'	It has to be tested to figure out if there are huge differences between GUIDE versions. So far we know only GUIDE 2026.1 supports debugging.

(continued on next page)

(continued from previous page)

ID	Disposition	Notes and Representative Quotes	Internal Notes
REQ-35	Future Work	P1: 'To convince the team, usually having a demo session and a go-to person in case of problems helps. Mostly they are not open to new things because they don't want a long learning. They want to use it as easy and fast as possible.'	
REQ-36	Partial (I1)	Flagged as minimum-viable by both interview participants. P10 minimum requirement: show a similar project tree structure as in GUIDE. P9: 'The project manager — so that I can manage the C code of the project without having to jump back to GUIDE to add and remove files.'	POUs not considered yet.
REQ-37	Delivered (I1)	P10 minimum: 'automatically update the `p1x` file when adding/removing C code files'. P9: when the task is purely in C he 'just want[s] to start VS Code and add or remove the stuff in the project'. Safety Partial because a corrupted `.p1x` could break the project.	
REQ-38	Future Work	P10 explicit: 'VS Code must be able to identify and notify about changes that has been made on the GUIDE side, and vice versa – it must be synchronized!' Extends REQ-14 with the VS Code ← GUIDE direction. Safety Partial because undetected GUIDE-side changes could overwrite reviewed C code.	Conflicting answers. Some say they want no pop-ups; some say they want indicators.
REQ-39	Future Work	P9: the current flow is 'not super intuitive… for a new user it is not super intuitive.' P10: adding C files is 'hidden in a context menu' and 'it's not easy to understand that headers must be made available for C Code POUs before they can be used.' Complements REQ-21 (single-action wrapper generation) by covering the broader onboarding flow and the header-availability step that REQ-21 does not address.	
REQ-40	Delivered (I2)	P9: 'since I'm getting older and my eyes aren't what they used to be, I need larger text.' GUIDE's current zoom/theming is described as dated. VS Code provides the base capability; the requirement is that the plugin respects it.	
REQ-41	Future Work	P4: GUIDE builds take 3 min and run 5–10×/hour. P6: up to 10 min for complex projects. P2: 15–20 min on bigger projects. Root cause lives inside GUIDE's compiler/SCS regeneration; the plugin's direct response is REQ-16 (C-only compile).	
REQ-42	Future Work	P4 Q8: 'But GUIDE does not offer a merge tool.' Teams currently split work across blocks/pages or rely on Git + GUIDE's SCS compare. Would require parsing GUIDE's proprietary SCS format.	

(continued on next page)

(continued from previous page)

ID	Disposition	Notes and Representative Quotes	Internal Notes
REQ-43	Future Work	P3 Q1: 'The best practice is to use C POUs as wrappers around your C function. This is the only part that I feel should not be necessary.' P3 notes this is 'coming in the future' from Danfoss. P5: 'just don't make me use the POU interface.' P10 reinforces from the interview. Plugin-side responses already exist as REQ-21 and REQ-39.	
REQ-44	Future Work	P4 Q3: 'The lack of folder structures is really annoying. If we want to use a library and it was written with folder structures, we need to flatten it.' Enforced by GUIDE's project model; plugin could at most display a virtual structure.	Might be already taken care of by VS Code.
REQ-45	Future Work	P7 Q12: 'The bool problem with Stdbool and GUIDE's NV bool struct name should be improved.' Resolution lives in GUIDE's type headers; plugin could add a lint rule that warns on the collision.	
REQ-46	Out of Scope	P6 Q12: 'GUIDE today does not support DBC file management for CAN communication.' P6 Q2 lists Vector DBC++ editor as the current workaround. Technically feasible as a VS Code extension, but outside the C/C++ inner-loop scope.	
REQ-47	Out of Scope	P6 Q12: 'GUIDE does not have an automated synchronization between GUIDE code pages and Service Tool diagnostic pages with namespace feature.' Requires Service Tool to expose a cooperating API.	
REQ-48	Future Work	P6 Q12: 'GUIDE does not support the auto-generation of the code documentation.' Well-supported by existing VS Code Doxygen extensions; cleanly in scope for a C/C++ plugin.	Should work with the Doxygen plugin in VS Code but has not been tested.
REQ-49	Future Work	P3 Q4: 'Biggest waste of time is that I can't see what powers/sets the value of a wire with one click, I might have to manually check 20+ pages until I see the source.' P4 Q4: traces wires across modules, reloading each one, which he describes as 'quite time consuming'. Requires parsing the proprietary SCS format.	

E.3 Pain-Point Evidence: Five Most-Cited Findings

Pain Point	Cited by	Req. ID	Representative Quote
Real-time diagnostics (no IntelliSense-equivalent feedback in the GUIDE C editor)	9/10	REQ-01, REQ-02	"GUIDE currently surfaces many errors only at build time, and builds can take 3–20 minutes." (Analyst note; consistent with all high-MoSCoW code-intelligence respondents.)
Ctrl+click navigation to definitions and references	8/10	REQ-04, REQ-05	"P7 ranks Ctrl+click to source as #1 missing feature." (Analyst note from requirements catalogue.)
Build latency (full-project build times 3–20 min at 5–10 invocations per hour)	Multiple	REQ-41	"GUIDE builds take 3 min and run 5–10×/hour." (P4); "up to 10 min for complex projects." (P6); "15–20 min on bigger projects." (P2).
File-synchronisation / manual reload friction	6/10	REQ-14	"[The current swap workflow:] remove, save, close GUIDE, reopen, copy, add, save." (P1, paraphrased from analyst note.)
Multi-step C Code POU wrapper procedure	5/10	REQ-21, REQ-39	"The best practice is to use C POUs as wrappers around your C function. This is the only part that I feel should not be necessary." (P3)

Table E.0.3: Representative evidence for the five most-cited Cycle I pain points.

F. Cycle I and Cycle II Informal Feedback Items

Informal feedback items from the Cycle I usability probe and Cycle II practitioner sessions are summarised below.

Cycle	Feedback Item	Description
Cycle I	GDB path must be specified in `launch.json`	GUIDE ships GDB 11.2 at a fixed installation path (`C:\Program Files (x86)\Danfoss\PLUS1\Base\1.0\GDB\bin\gdb.exe`). The Cycle I probe participant noted that this path must be declared in the debug configuration; the extension cannot assume a system-wide GDB installation.
Cycle I	No CLI command to generate a debugged application	At the time of Cycle I, PLUS+1 GUIDE had no command-line interface entry point for generating the debugged executable (`.exe`) that GDB operates on. The Debug button inside GUIDE triggers generation and places the output in a `Debug/` subfolder, but this was not scriptable from outside GUIDE.
Cycle I	PLUS+1 activity-bar icon requested as highest-priority visual feature	The Cycle I probe identified a custom PLUS+1 icon in the VS Code activity bar as the top-priority visual affordance, making the extension immediately distinguishable from standard VS Code panels.
Cycle I	Diagnostic-parameter grid view	Engineers add diagnostic variables (Checkpoint, Set Value, Set Pulse, Retain) to PLC code through GUIDE's interface. A participant requested a structured grid view in VS Code where the type, name, and data type of each diagnostic variable can be added or removed, with changes written back to `PLCFile1.xml`.
Cycle I	Project creation from VS Code	A participant requested the ability to create a new PLUS+1 GUIDE project from inside VS Code via a command that calls `Plus1_IDE.exe -create-project <hwd_partnumber> <p1x_file> -use-template -headless`. For the proof-of-concept it was noted that supporting a single hardware target would be sufficient.
Cycle II	Auto-save C files on compile or debug	A participant noted that unsaved C files are not picked up by the headless build unless saved first. The expected behaviour is that triggering a build or debug session saves all modified files automatically.
Cycle II	Automatic detection of the GUIDE installation path	In the initial prototype, the user was required to edit a settings entry manually to supply the path to `Plus1_IDE.exe` before the first build. A participant identified this as an adoption barrier.
Cycle II	Project tree not refreshed after a successful build	After a headless build, newly generated `.lhx` hex output files were not visible in the project tree without a manual refresh.
Cycle II	`.lhx` deletion corrupts the `.p1x` file	Deleting an `.lhx` file from the project tree incorrectly removed the corresponding `<Hexfile>` XML element from the `.p1x` project file, corrupting the project. The `<Hexfile>` element is required by GUIDE and must always be present.

(continued on next page)

(continued from previous page)

Cycle	Feedback Item	Description
Cycle II	Files in the `.sys` folder must not be renamable or deletable	A participant noted that the `.sys` folder contains system-managed files that must not be modified through the extension. Allowing rename or delete operations on these files could silently corrupt the GUIDE project.
Cycle II	`.scs`, `.xml`, and `.json` files must not appear in the project tree	The project tree initially displayed all file types found under the project directory, including SCS graphical-program files, XML configuration files, and JSON build artefacts. A participant requested that these be hidden, as they are not relevant to the C/C++ editing workflow and clutter the view.
Cycle II	Build diagnostics visible only at task completion	Under the initial `cp.spawn`-based build implementation (ADR-006), diagnostic output was written to a custom `DiagnosticCollection` only after the build process exited, meaning errors were not visible in the Problems panel until the full build completed. A participant noted this prevented early identification of errors in long builds.
Cycle II	`.clangd` written to `.vscode/` not discovered by `clangd`	The initial IntelliSense configuration design (ADR-008) wrote the `.clangd` YAML file to the `.vscode/` subdirectory and populated `c_cpp_properties.json` by scraping the Makefile produced by a successful build. Testing revealed that `clangd`'s configuration search walks up the directory tree from each source file and does not find a `.clangd` file inside `.vscode/`. Additionally, Makefile scraping meant IntelliSense was unavailable before the first successful build.

G. Cycle III SUS Item-Level Results

Table G.0.1: Cycle III SUS Item-Level Responses

SUS Item	P1	P2	P3	P4	P5	Mean
1. I think that I would like to use this system frequently.	3	5	5	5	4	4.40
2. I found the system unnecessarily complex.	4	1	2	2	2	2.20
3. I thought the system was easy to use.	3	5	4	4	5	4.20
4. I think that I would need the support of a technical person to be able to use this system.	4	1	3	3	2	2.60
5. I found the various functions in this system were well integrated.	2	5	4	5	4	4.00
6. I thought there was too much inconsistency in this system.	4	2	2	1	2	2.20
7. I would imagine that most people would learn to use this system very quickly.	3	4	4	4	4	3.80
8. I found the system very cumbersome to use.	4	1	1	1	2	1.80
9. I felt very confident using the system.	3	5	4	3	4	3.80
10. I needed to learn a lot of things before I could get going with this system.	4	1	2	1	2	2.00
SUS Score	**35.0**	**95.0**	**77.5**	**82.5**	**77.5**	**73.5**

Mean: 73.5, Median: 77.5, Standard Deviation: 22.7

H. Cycle III Structured Comparison Responses

Table H.0.1: Cycle III Structured Comparison Item Responses (C1–C8)

Item	Focus	P	Response
C1	Overall comparison	P1	I feel like it is restricted because it cannot work with POUs, CCPs, SDLs, HWDs (Hardware compliance).
		P2	Navigating between C files, finding variable references, go to definitions, adding C files improved the workflow very much.
		P3	The modern look and feel of VS Code, with features expected from a code editor.
		P4	Not having to jump between GUIDE and another text editor is great and saves a lot of time.
		P5	I would say it definitely is more streamlined in this extension than in GUIDE. It is very nice to have a more modern interface.
C2	Pain-point resolution	P1	Not really.
		P2	Yes, all the main issues were addressed.
		P3	Yes, specifically debugging, and code naviation.
		P4	Yes, see C1.
		P5	Yes, it felt like a much better user experience and has the features I would expect from a modern editor.
C3	Unresolved limitations	P1	I think it is difficult to know what the full intentions of the extension are. So therefore, difficult to know what to expect from it.
		P2	Editing POU interface, I did not expect to have it already now but would be nice in the future.
		P3	The build process should be faster, and also debugger should start faster.
		P4	No.
		P5	I've never used only C code in a GUIDE application. So I'm not sure how this would work with a workflow that was mixed C and
C4	VS Code platform benefits	P1	The right click options did not appear to work. Also compiling from the Run menu does not appear to work.
		P2	Yes, I also used the integrated GitHub Copilot agent which improved the workflow.
		P3	Copilote.
		P4	Many projects are worked on by more than one person and they have to agree on one GUIDE version to use. Currently, there is
		P5	It all was beneficial. Even having used VSCode for a while, I'm still getting used to the top quick access bar and how to use it.
C5	Adoption intent	P1	I think this would only be useful if it allowed better flow between the graphical and text-based language. We are not working on
		P2	Yes! The already included features are good enough for me to adopt it regularly. I would consider stop using it if I would need t
		P3	Yes. Being able to have GUIDE open, faster build / debug build, source in folders.
		P4	Yes, I will use that for projects with a lot of textual code.
		P5	Yes. I would need C++ also supported and for this to work with Linux based controllers (ex. XM100).
C6	Top improvement	P1	Allow integration of CCP, POU, SDL and HWD for reuse.
		P2	Debugger (although it is already part of the POC).

(continued on next page)

(continued from previous page)

Item	Focus	P	Response
		P3	Faster build.
		P4	Making it compliant with IntelliSense.
		P5	It would be very nice to be able to view the possible options for autocompleting to select from, specifically for inputs and outp
C7	Missing features	P1	Need to publish the intended use cases and better help so users know how to use it.
		P2	1. Possibility to edit POU interface (add/remove/modify variables). 2. Create new GUIDE project from scratch in VS Code.
		P3	Create new projects from VS code.
		P4	See C4.
		P5	Not that I can think of.
C8	Open comment	P1	No.
		P2	No.
		P3	Great work!
		P4	—
		P5	—